

Hierarchical Abstract Tree for Cross-Document Retrieval-Augmented Generation

Ziwen Zhao¹ Menglin Yang¹

Abstract

Retrieval-augmented generation (RAG) enhances large language models with external knowledge, and tree-based RAG organizes documents into hierarchical indexes to support queries at multiple granularities. However, existing Tree-RAG methods designed for single-document retrieval face critical challenges in scaling to cross-document multi-hop questions: (1) *poor distribution adaptability*, where k -means clustering introduces noise due to rigid distribution assumptions; (2) *structural isolation*, as tree indexes lack explicit cross-document connections; and (3) *coarse abstraction*, which obscures fine-grained details. To address these limitations, we propose Ψ -RAG, a tree-RAG framework with two key components. *First*, a hierarchical abstract tree index built through an iterative “merging and collapse” process that adapts to data distributions without a priori assumption. *Second*, a multi-granular retrieval agent that intelligently interacts with the knowledge base with reorganized queries and an agent-powered hybrid retriever. Ψ -RAG supports diverse tasks from token-level question answering to document-level summarization. On cross-document multi-hop QA benchmarks, it outperforms RAPTOR by 25.9% and HippoRAG 2 by 7.4% in average F1 score¹.

1. Introduction

Retrieval-augmented generation (RAG) (Lewis et al., 2020) enhances large language models (LLMs) with reliable references from external knowledge bases. To effectively address multi-granular user queries from token-level factual

¹The Hong Kong University of Science and Technology (Guangzhou), Guangzhou, China. Correspondence to: Menglin Yang <menglinyang@hkust-gz.edu.cn>.

Proceedings of the 43rd International Conference on Machine Learning, Seoul, South Korea. PMLR 306, 2026. Copyright 2026 by the author(s).

¹Code is available at <https://github.com/Newiz430/Psi-RAG>.

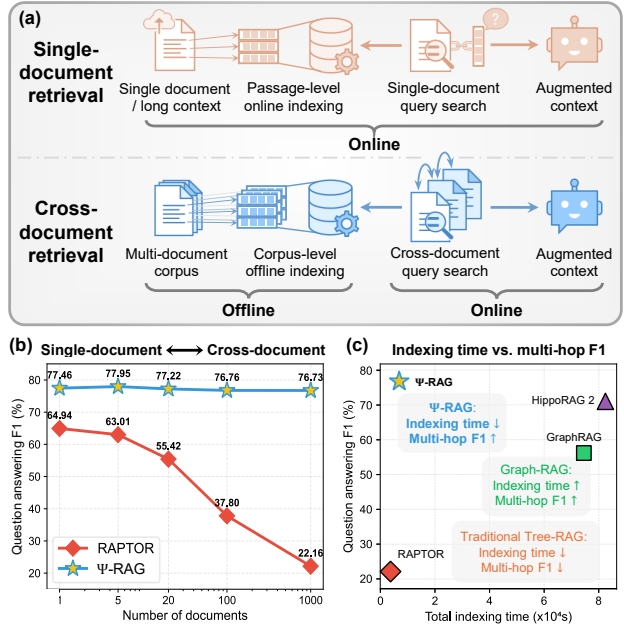


Figure 1. (a) Two RAG application scenarios: single-document and cross-document retrieval. (b) Multi-hop QA performance w.r.t. the number of documents in the tree index. (c) Ψ -RAG has both efficient indexing and accurate multi-hop retrieval compared to structured RAG baselines, using 2Wiki as an example.

question answering (QA) to document-level summarization, RAG frameworks must leverage the semantic hierarchy within corpora (Sarathi et al., 2024; Chen et al., 2024). Traditional retrieval systems (Robertson et al., 2009; Karpukhin et al., 2020) slice documents into short, independent chunks; this fixed retrieval granularity leads to incomplete and inaccurate responses for higher-level questions (Nair et al., 2023; Liu et al., 2021; Sarathi et al., 2024; Wang et al., 2026).

Structured RAG achieves multi-granularity by organizing documents relationally with knowledge graph indexes (Edge et al., 2024; Jiménez Gutiérrez et al., 2024; 2025) or hierarchically with tree indexes (Liu et al., 2021; Jin et al., 2025b; Sarathi et al., 2024). Its utility is demonstrated in two key scenarios, as in Figure 1(a): (1) *single-document retrieval*, to process user requests in diverse granularity within a single user-uploaded document (Jin et al., 2025a; Li et al., 2025b; Zhang et al., 2025a); (2) *cross-document retrieval*, to process user requests by building a large index from an entire

Table 1. Comparison of RAG frameworks. “?” indicates limited support by specific methods.

Task type	Single-document	Cross-document	Token-level	Passage-level	Document-level
Traditional RAG	✓	✗	✓	?	✗
Graph-RAG	✓	✓	✓	✓	?
Tree-RAG	✓	✗	?	✓	✓
Ψ-RAG (Ours)	✓	✓	✓	✓	✓

(domain-specific) corpus and retrieving across multiple documents, challenging the multi-hop retrieval and reasoning capabilities of RAG frameworks (Trivedi et al., 2023; Tang & Yang, 2024).

Our work focuses on *Tree-RAG* (Liu et al., 2021; Jin et al., 2025b; Sarthi et al., 2024; Tao et al., 2025; Rezazadeh et al., 2025), which organizes documents into a tree-structured index. It explicitly defines different information granularities, enabling superior retrieval performance, especially for summative tasks (Sarthi et al., 2024; Xiao et al., 2025). See Appendix F for more related works. However, existing Tree-RAG methods mainly target single-hop retrieval on a single long document, while cross-document and multi-hop retrieval remain underexplored. For example, representative *k*-means-type cluster trees like RAPTOR (Sarthi et al., 2024) achieve high retrieval performance only with small-scale, passage-level indexes. As shown in Figure 1(b), **RAPTOR’s retrieval accuracy drops significantly when the search space expands to corpus-level with millions of tokens.** We identify three key limitations: (1) *Poor distribution adaptability*: *k*-means-type clustering implicitly relies on spherical data distributions (Sarthi et al., 2024), which introduces noisy documents to the retriever for corpora with a skewed distribution. (2) *Structural isolation*: unlike Graph-RAG which dynamically hops between documents based on pairwise relationships, leaf nodes in a tree index lack explicit connections. This prevents the retriever from capturing implicit causal dependencies in multi-hop questions. (3) *Coarse abstraction*: the coarse-grained abstracts act like a mosaic, obscuring token-level details at the very beginning of retrieval, as dense vector matching struggles to precisely associate a specific entity in the user query with abstract concepts on top of the tree.

This work aims to address the limitations of Tree-RAG on cross-document and multi-hop retrieval. We present Ψ-RAG, a versatile and effective Tree-RAG framework, comprising a *hierarchical abstract tree index* and a *multi-granular agentic retriever*. (1) To address poor distribution adaptability, we build an abstract tree index via a hierarchical clustering-inspired “merging and collapse” process, which iteratively links similar chunks to existing or newly generated upper-level nodes in the tree. (2) To tackle structural isolation, a retrieval and answering agent reasons through the user query, during which it calls for additional

document retrieval with a reorganized query when needed. This empowers the tree retriever with better causal understanding of user query. (3) To resolve the conflict between coarse abstraction and detailed factual retrieval, a multi-granular hybrid retrieval framework is employed, where an agent-powered sparse retriever supplements the hierarchical tree retrieval with fine-grained information, yielding significant performance gains. Our proposed framework possesses numerous advantages:

- **Ψ-RAG generalizes Tree-RAG from passage-level to corpus-level indexing.** As shown in Figure 1(c), Ψ-RAG builds a corpus-level tree 10× faster than OpenIE-based Graph-RAG. Under this setting, Ψ-RAG achieves average gains of 23.7% in retrieval and 25.9% in generation over RAPTOR on token-level QA.
- **Ψ-RAG extends Tree-RAG to complex multi-hop scenarios.** To our knowledge, Ψ-RAG is the first Tree-RAG for cross-document multi-hop scenarios comparable to Graph-RAG, exhibiting great application potential. As in Figure 1(c), Ψ-RAG is both efficient and powerful, outperforming cutting-edge Graph-RAG frameworks like HippoRAG 2 (Jiménez Gutiérrez et al., 2025).
- As in Table 1, **Ψ-RAG is an all-in-one Tree-RAG framework supporting tasks across various granularities**: token-level factual QA, passage-level causal reasoning, and document-level summarization.
- **Ψ-RAG is built entirely with open-source LLMs.** Its components are flexibly replaceable. Ψ-RAG can be applied to custom corpora without any training or fine-tuning, demonstrating strong generalizability.

2. Preliminary

Retrieval-Augmented Generation. Given a user query q , an LLM $f(q; I)$ generates a corresponding response a , where I is the system instruction for a specific task. For a reasoning model, $a = R \cup y$, where R is the Chain-of-Thought (Wei et al., 2022) reasoning text and y is the final answer. An LLM equipped with RAG can be defined as $f \cdot r$, where the retriever $r(q; \mathcal{I})$ maps q to top- k relevant document chunks $\mathcal{D}^* = \{u_i\}_{i=1}^k \subset \mathcal{D}$ using a scoring function $s(q, u)$. The external corpus $\mathcal{D}$ contains $n = |\mathcal{D}|$ chunks in total. The index $\mathcal{I} : \mathcal{D} \rightarrow \mathcal{T}$ maps each chunk in $\mathcal{D}$ to a discrete index space $\mathcal{T}$ for better retrieval.

Tree Index. We overload the symbol $\mathcal{T}$ to denote a tree. Formally, a tree $\mathcal{T} = (\mathcal{V}, \mathcal{E})$ is a connected acyclic graph with a finite node set $\mathcal{V} = \{v_i\}_{i=1}^{|\mathcal{V}|}$ and an edge set $\mathcal{E} \subset \mathcal{V} \times \mathcal{V}$. The root node for a node u is denoted as $root(u)$. The set of children of u is $c(u) = \{v | u \rightarrow v \in \mathcal{E}\}$. Let $\ell(u)$ be the set of all leaf nodes reachable from u . The depth of a node $\delta(\cdot)$ is defined as the length of the path to the root,

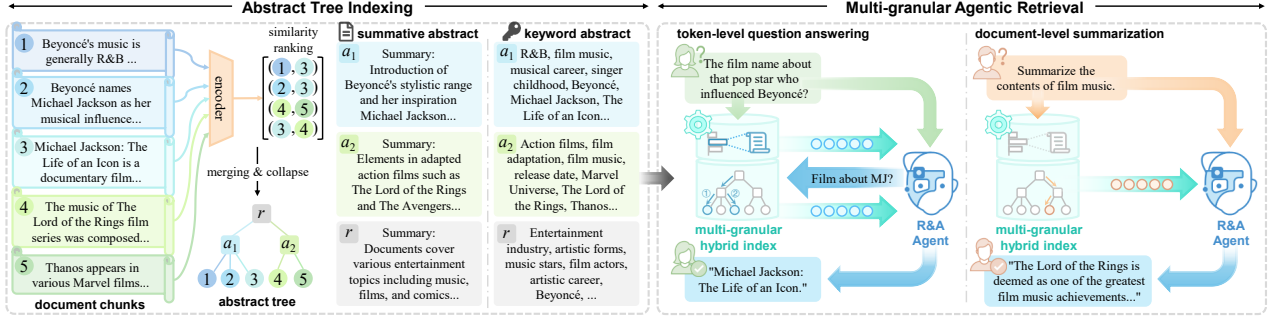


Figure 2. Overview of Ψ -RAG. In Abstract Tree Indexing, a hierarchical abstract tree is built by ranking pairwise chunk similarities and applying an iterative “merging and collapse” process. Then, an abstraction agent generates a summative or keyword abstract for each abstract node. In Multi-granular Agentic Retrieval, a retrieval and answering (R&A) agent dynamically interacts with a multi-granular knowledge base organized by both the tree index and a sparse keyword index. By adjusting the retrieval iterations and the utilization of the sparse index, Ψ -RAG adapts to diverse user requests from token-level question answering to document-level summarization.

with $\delta(\text{root}) = 0$. The tree depth is $\delta(\mathcal{T}) = \max_{u \in \mathcal{T}} \delta(u)$. We denote the parent (ancestor) of leaf u at a distance k as $p^k(u)$, where $p^0(u) = u$ and $\text{root}(u) = p^{\delta(u)}(u)$. The least common ancestor (LCA) of two nodes is defined as the deepest node that is an ancestor to both:

$$\text{LCA}(u, v) = \arg \max_{a \in \{p^k(u)\}_{k=1}^{\delta(u)} \cap \{p^k(v)\}_{k=1}^{\delta(v)}} \delta(a) \quad (1)$$

A tree is binary (or bifurcating) if all its non-leaf nodes have exactly two children. A tree is multifurcating if all non-leaf nodes have at least two children and at least one node has more than two children. A tree is equi-depth if every leaf is equidistant from the root.

3. Ψ -RAG

As illustrated in Figure 2, Ψ -RAG follows the two-phase paradigm of existing structured RAG: indexing and retrieval. This section formally presents its two core components: Abstract Tree Indexing and Multi-granular Agentic Retrieval.

3.1. Abstract Tree Indexing

Ψ -RAG constructs a tree index from a long document or a document collection, which we term an *abstract tree*. The leaves consist of the original document chunks $\mathcal{D}$ and their encoded dense vectors. Upper-level nodes are called *abstract nodes*, each containing an abstract of its children generated by an abstraction agent. The abstract tree is constructed via a “similarity ranking $\rightarrow$ iterative merging and collapse $\rightarrow$ abstraction” pipeline.

Similarity ranking. First, all document chunks are encoded into fixed-length dense vectors $e(\mathcal{D}) \in \mathbb{R}^{|\mathcal{D}| \times d_e}$ using a pre-trained language encoder. A symmetric similarity matrix $\mathbf{S} \in \mathbb{R}^{|\mathcal{D}| \times |\mathcal{D}|} = e(\mathcal{D})e(\mathcal{D})^\top$ is then constructed. These chunk pairs are finally sorted in descending order based on their similarity scores.

Iterative merging and collapse. Beginning from the most

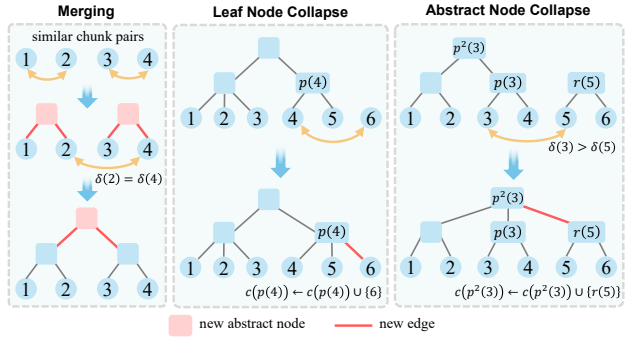


Figure 3. Illustration of abstract tree building steps: merging, leaf node collapse, and abstract node collapse. Yellow arrows indicate the current chunk pair to be processed.

similar pair of document chunks (u, v) , the tree is built iteratively: if neither u nor v has a parent, they are linked under a newly created abstract node a : $c(a) = \{u, v\}$. This is called a *merging*. If one node u is already linked to an abstract node $p(u)$, the isolated node v is also linked under $p(u)$: $c(p(u)) \leftarrow c(p(u)) \cup \{v\}$. This is called a *leaf node collapse*. If both u and v are already linked to abstract nodes and their roots differ, then:

- If $\delta(u) = \delta(v)$, their roots are merged under a new abstract node a : $c(a) = \{\text{root}(u), \text{root}(v)\}$, making $\text{root}(u) = \text{root}(v) = a$.
- If $\delta(u) > \delta(v)$, $\text{root}(v)$ is linked onto the path of u :

$$c(p^{\delta(v)+1}(u)) \leftarrow c(p^{\delta(v)+1}(u)) \cup \{\text{root}(v)\} \quad (2)$$

This results in $\delta(u) = \delta(v)$. The case for $\delta(u) < \delta(v)$ is analogous. This is called an *abstract node collapse*.

Figure 3 illustrates these tree-building steps. The iteration runs for exactly $n - 1$ steps, which is the minimum number of connections required to link all n nodes into a single tree. Subsequently, the abstract tree is *rebalanced* by splitting

nodes with an excessive number of children and distributing the children evenly between two new nodes. This avoids overly long contexts for the abstraction agent. A detailed algorithm is provided in Appendix A.

Abstraction. This step employs an abstraction agent to generate and encode an abstract for each abstract node. We consider two abstraction types. (1) *Summative abstract*: paragraphs of summary text (Sarathi et al., 2024; Edge et al., 2024) that capture coherent logic and key entity relationships. (2) *Keyword abstract*: keywords of different types distilled from the document chunks, with high information density but weak logical correlations. Phrases are encouraged over single words to preserve the entity co-occurrence.

3.2. Multi-granular Agentic Retrieval

Existing Tree-RAG frameworks struggle with multi-hop questions due to the structural isolation of tree indexes. For a multi-hop query like “Who is the wife of the man (David Gest) who produced the documentary of the pop star who influenced Beyoncé?”, a retriever relying solely on dense vector similarity would prioritize content related to “Beyoncé” and “documentary”, while missing the true subject “David Gest”. As such implicit associations often follow a causal order, existing tree retrieval methods like beam search (Sarathi et al., 2024; Gupta et al., 2025) are insufficient for ordered causality reasoning.

Retrieval and Answering (R&A) Agent. To capture ordered causality during reasoning, we employ a versatile R&A agent that intelligently engages in multi-turn interactions with the abstract tree index. It first assesses whether the available information is sufficient to answer the user query accurately or whether an additional retriever call is necessary. The agent response is a triple $a = (R, \langle \text{action} \rangle, \cdot)$ where the action tag is either $\langle \text{answer} \rangle$ or $\langle \text{retrieve} \rangle$. If $\langle \text{retrieve} \rangle$ is returned, the agent proposes a new query q' and requests another retrieval. The newly retrieved documents are then fed back to the agent alongside the original user query for the next reasoning step.

For each retrieval step, Ψ -RAG performs top-down dense vector matching following (Sarathi et al., 2024). Starting from the root, it computes the cosine similarity $s(q, u)$ between the query vector and each candidate node at layer l , selecting the top- k nodes $\mathcal{U}_l$. Then, all children of the selected nodes $\bigcup_{u \in \mathcal{U}_l} c(u)$ become the candidates for the next layer. This process iterates until the leaf layer is reached.

Our agentic retrieval workflow is as follows. Given a user query q , an initial retrieval is invoked. The returned top- k chunks $\mathcal{D}_0^*$ are fed to the agent, yielding $a_1 = f(q_0; I(\mathcal{D}_0^*))$. Then, “retrieval-generation” steps are iterated:

$$\mathcal{D}_i^* = r(q_i', \mathcal{T}), a_{i+1} = f(q; \bigcup_{j=1}^i (I(\mathcal{D}_j^*) \cup a_j) \cup I(\mathcal{D}_0^*)) \quad (3)$$

until the action tag becomes $\langle \text{answer} \rangle$ or the number of retrieval steps reaches a threshold $i_{\max}$. The user instruction I continuously reminds the agent of the remaining retrieval attempts. If the information remains insufficient after exhausting all attempts, the R&A agent returns “Not mentioned”.

Agent-powered Hybrid Retrieval. In a corpus-level search space, the coarse-grained upper-level abstracts make even simple factual searches challenging for Tree-RAG. To better capture fine-grained information, a sparse keyword index is constructed during tree indexing using BM25 (Robertson et al., 2009). During retrieval, the R&A agent combines results from both the tree and sparse indexes via two optional strategies: (1) parametrically, using a reranker model to integrate both top- k chunks; or (2) non-parametrically, via reciprocal rank fusion (RRF) (Cormack et al., 2009).

Furthermore, the R&A agent broadens the scope of sparse retrieval. It is encouraged to reorganize its query with enriched context, such as inserting descriptive appositions:

$$q : \text{“Who is the wife of David Gest?”} \xrightarrow{\text{reorganization}} q' : \text{“Who is the wife of the American film producer David Gest?”}$$

This adds thematic keywords for the sparse retriever and, more importantly, aids the tree retriever in locating relevant upper-level abstract nodes with the high-level context. An algorithm of the agentic retrieval is given in Appendix A.

4. Distribution Adaptability of Tree-RAG

This section analyzes the distribution adaptability of k -means-type Tree-RAG and Ψ -RAG. Tree-RAG methods like RAPTOR (Sarathi et al., 2024) build an equi-depth tree index via bottom-up k -means-type clustering, such as the Gaussian Mixture Model (GMM). However, k -means-type clustering tends to assign objects from major clusters to minor ones, resulting in clusters of relatively uniform size even when the corpus follows a skewed distribution. This is known as the “uniform effect” (Xiong et al., 2006; Liang et al., 2012; Assran et al., 2023). For RAG, this effect may cause the retriever to overlook chunks with minor themes. In contrast, Ψ -RAG better preserves skewed distributions in corpora. We compare the two tree RAG frameworks through the lens of their optimization objectives.

4.1. The Uniform Effect

k -means-type tree index. A cluster can be defined as a family of leaf sets $\mathcal{C} = \{c = \ell(\mathcal{T}_c)\}$ from an equi-depth forest $\{\mathcal{T}_c \subseteq \mathcal{T} \mid \delta(\mathcal{T}_c) = \delta(\mathcal{T}) - 1\}$. A k -means-type cluster tree searches for an optimal $\mathcal{C}$ by minimizing the objective

$$\mathcal{L} = \sum_{i=1}^k \sum_{x_i \in c_i} \|x_i - \mu_i\|^2, \quad (4)$$

where $k = |\mathcal{C}|$, $\mu_i = \sum_{x_i \in c_i} \frac{x_i}{n_i}$ is the cluster centroid and $n_i = |c_i|$. Theorem 4.1 presents a variant of (4) that reveals the “uniform effect” in k -means clustering.

Theorem 4.1 (Uniform effect of k -means (Liang et al., 2012)). *Given $\mathcal{L}$ in (4), we have*

$$\mathcal{L} = -A \sum_{1 \leq i < j \leq k} n_i n_j \|\mu_i - \mu_j\|^2 + B \quad (5)$$

where A and B are constants dependent only on the data.

See Section B.1 for the proof. Assuming $n_i n_j$ and $\|\mu_i - \mu_j\|^2$ are independent, maximizing the term $n_i n_j$ in (5) leads to $n_1 = n_2 = \dots = n_k = n/k$. This indicates that k -means favors clusters of uniform size. (Proposition 1, (Assran et al., 2023)) further notes that soft k -means, i.e., GMM, shares the same global optimum and the uniform effect with k -means. Figure 4 shows a toy example where k -means gradually shifts the minor cluster centroid toward the major class, inflating the minor cluster beyond its actual size. This may confuse abstract nodes in a tree index which leads to semantic bias or neglect of minor nodes.

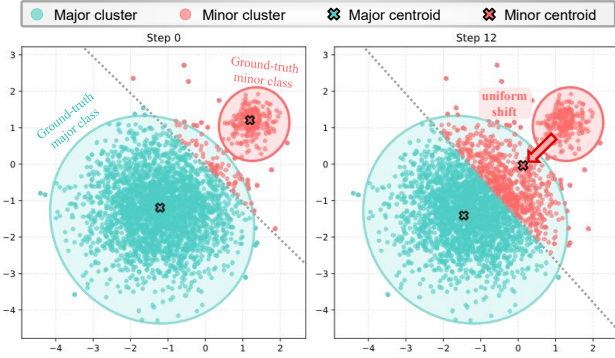


Figure 4. The uniform effect of k -means clustering. Initial centroids are set at the center of each class (step 0). As iteration proceeds, more major class nodes are misassigned to the minor cluster, eventually shifting the minor centroid (step 12).

Ψ -RAG’s hierarchical index. Ψ -RAG is inspired by agglomerative hierarchical clustering (AHC) (Florek et al., 1951; Sokal & Michener, 1958), a greedy heuristic algorithm without an explicit optimization objective. We therefore adopt a widely used metric for evaluating AHC.

Definition 4.2 (Dasgupta’s cost (Dasgupta, 2016)). For a tree $\mathcal{T}$, the Dasgupta’s cost is defined as

$$c(\mathcal{T}) = \sum_{1 \leq i < j \leq n} d(u_i, u_j) |\ell(\text{LCA}(u_i, u_j))| \quad (6)$$

where $d(\cdot, \cdot)$ is a distance function.

The intuition is that semantically similar points (with small d) should have a deeper LCA, i.e., be placed closer in the tree. Dasgupta’s cost is widely regarded as the implicit

minimization objective of AHC (Moseley & Wang, 2017; Charikar & Chatziafratis, 2017; Chami et al., 2020).

Based on Dasgupta’s cost, we analyze two scenarios: (1) whether Ψ -RAG tends to build a tree with uniform leaf distribution, and (2) whether it adaptively generates subtrees of varying sizes for minor clusters. Our findings are formalized in the following Theorems.

Theorem 4.3 (Non-uniformity preference of Ψ -RAG). *Consider a 3-layer equi-depth multifurcating tree $\mathcal{T}$ with a perfectly uniform leaf distribution among its subtrees $\mathcal{T}_1, \dots, \mathcal{T}_k$: $n_1 = n_2 = \dots = n_k = n/k$, where $k \geq 3$, $n \geq 3k$. A new tree $\mathcal{T}'$ is constructed by moving one leaf from $\mathcal{T}_j$ to another subtree $\mathcal{T}_i$. Then:*

- (1) $c(\mathcal{T}') < c(\mathcal{T})$;
- (2) Let $\mathcal{T} = \psi(\mathcal{T}_{\text{AHC}})$ and $\mathcal{T}' = \psi(\mathcal{T}'_{\text{AHC}})$, where $\psi(\cdot)$ denotes the collapsing process from a binary tree to an equi-depth multifurcating tree. Denote by Δc and $\Delta c'$ the increments of Dasgupta’s cost on $\mathcal{T}$ and $\mathcal{T}'$ introduced by $\psi(\cdot)$, respectively. Then we have $c(\mathcal{T}') - c(\mathcal{T}) = \Delta c' - \Delta c$.

Theorem 4.4 (Distribution adaptability of Ψ -RAG). *Consider a 3-layer equi-depth multifurcating tree $\mathcal{T}$ with subtrees $\mathcal{T}_1, \dots, \mathcal{T}_k$ of any size where $k \geq 3$. A subtree of a minor leaf cluster $\mathcal{T}_i$ has $2 \leq n_i < n_j, \forall 1 \leq j \neq i \leq k$. A new tree $\mathcal{T}'$ is constructed by moving one leaf from some subtree $\mathcal{T}_j$ to the minor subtree $\mathcal{T}_i$. Then:*

- (1) $c(\mathcal{T}') \geq c(\mathcal{T})$, and $c(\mathcal{T}') = c(\mathcal{T})$ iff $n_j = n_i + 1$;
- (2) Let $\mathcal{T} = \psi(\mathcal{T}_{\text{AHC}})$, $\mathcal{T}' = \psi(\mathcal{T}'_{\text{AHC}})$ and $\Delta c, \Delta c'$ defined as in Theorem 4.3. Then we have $\Delta c' \geq \Delta c$, and $\Delta c' = \Delta c$ iff $n_j = n_i + 1$.

See Section B.2 and Section B.3 for the proofs and remarks. The above theorems prove that Ψ -RAG incurs a larger increment in Dasgupta’s cost for (1) a tree with perfectly uniform leaf distribution, and (2) a tree where nodes from major clusters are assigned to a minor cluster. They show that Ψ -RAG does not suffer from the uniform effect and, furthermore, adaptively preserves skewed distributions within corpora.

4.2. Visualization Study on Skewed Data

We conduct a visualization study by constructing tree indexes with both RAPTOR and Ψ -RAG on skewed datasets to validate the above conclusions. Three skewed datasets are generated from MultiHop-RAG (Tang & Yang, 2024): each combines the first 5 documents from one of three categories: “Business”, “Entertainment”, or “Technology”, with the first 50 documents from the semantically distant “Sports” class, as shown in Figure 5(a). We extract the bottom four layers of each tree and visualize them as circular trees.

The results in Figure 5(b) reveal that major abstract nodes in RAPTOR tend to be confused with (i.e., share a parent with) minor ones especially at higher layers, which is a manifestation of the uniform effect. In contrast, Ψ -RAG assigns

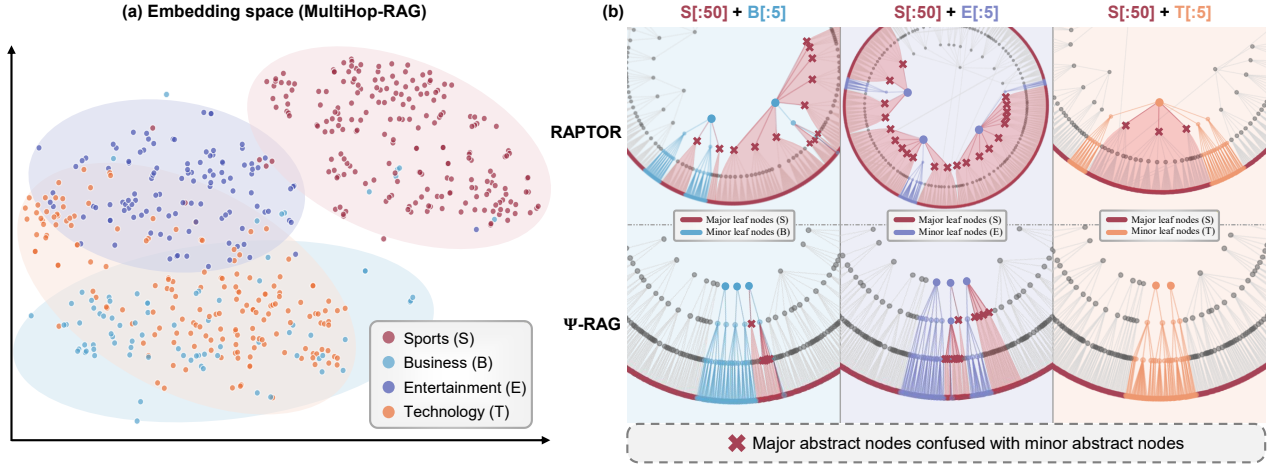


Figure 5. RAPTOR vs. Ψ -RAG on skewed distributions. (a) 2-dimensional embedding space of the four main categories of MultiHop-RAG. (b) Circular tree visualizations (only regions containing minor class nodes are shown). The outermost ring represents leaf nodes. For $S[:50] + B[:5]$, red points represent “Sports” nodes, while blue points and edges trace root-to-leaf paths for “Business” nodes. The red shaded area indicates the retrieval noise introduced by confused nodes (X). Comparing the number of confused nodes and the size of the shaded areas, Ψ -RAG organizes minor classes more effectively.

far fewer major abstract nodes to minor ones, resulting in an intuitively superior index structure. A detailed case study of the confused abstracts and its negative effect on retrieval is provided in Appendix D.2.

5. Experiments

5.1. Experimental Setup

Tasks and evaluation metrics. Our evaluation encompasses four types of tasks. (1) Single-hop and multi-hop QA (token-level fact answering) involves detailed facts within one or multiple passages under a *cross-document setting*. A word or a phrase is required as an answer. We use Recall@ k ($k = 2, 5$) for retrieval metrics and use Exact Match (EM) and F1 score for QA. (2) Narrative QA (passage-level causal reasoning) requires the model to identify causal relationships within a *single long document* for a narrative answer. Given the variable answer length, we employ F1 and the Recall-Oriented Understudy for Gisting Evaluation using the Longest Common Subsequence (ROUGE-L) as metrics. (3) Document-level summarization requires a comprehensive understanding of a *single long document* in response to a fixed query: “Summarize the contents of this meeting / news event”. Iterative retrieval and the sparse index are disabled for this task. Given the variable answer length and format, we use ROUGE-L as the metric.

Datasets. Each task is evaluated on multiple benchmark datasets. For single-hop and multi-hop QA, we mainly follow HippoRAG 2 (Jiménez Gutiérrez et al., 2025) and use 1,000 randomly sampled queries from (1) single-hop: NQ (Kwiatkowski et al., 2019) and PopQA (Mallen et al., 2023); (2) multi-hop: HotpotQA (Yang et al., 2018), 2Wiki (2Wiki-

MultihopQA) (Ho et al., 2020), and MuSiQue (Trivedi et al., 2022). We also include the first 1,000 questions from MultiHop-RAG (Tang & Yang, 2024), a multi-hop benchmark containing negative samples. For narrative QA, 10 long documents are randomly sampled from NarrativeQA (Kočísky et al., 2018). We also use the first 100 documents from the LongBook subset in ∞ Bench (Zhang et al., 2024a), a narrative QA dataset featuring significantly longer documents. For summarization, we adopt QMSum (Zhong et al., 2021) and WCEP (Ghalandari et al., 2020). Overly short documents are filtered out following (Zhang et al., 2025a). Further dataset details are provided in Appendix C.

Baselines. Our baselines cover various RAG paradigms. (1) Traditional RAG: the sparse retriever BM25 (Robertson et al., 2009), the dense vector retriever DPR (Karpukhin et al., 2020) with Faiss (Douze et al., 2025), and the strong hybrid retriever BM25 + DPR equipped with a reranker. Another baseline involves DPR equipped with an iterative agentic retriever IRCot (Trivedi et al., 2023). (2) Graph-RAG: GraphRAG from Microsoft (Edge et al., 2024), the state-of-the-art framework HippoRAG 2² (Jiménez Gutiérrez et al., 2025), and GoR (Zhang et al., 2025a), a new summarization-exclusive Graph-RAG framework. (3) Tree-RAG: RAPTOR (Sarthi et al., 2024), the only open-source framework that can be technically implemented to a corpus-level index.

Implementation details. Ψ -RAG (and all dense retrieval baselines) by default uses Qwen3-Embedding-8B (Q3-8B) (Zhang et al., 2025b) as the encoder and Llama-3.3-70B (L3-70B) (Dubey et al., 2024) as the abstraction and R&A agent. We use BM25S (Lù, 2024) as the BM25 implementation.

²We report both the original setting (with NV-Embed-V2 (Lee et al., 2025)) and our setting with Q3-8B and Q3R.

Hierarchical Abstract Tree for Cross-Document Retrieval-Augmented Generation

Table 2. Single- and multi-hop QA results in EM (%) and F1 (%). Best results are in **bold**. Second-bests are underlined.

Type	Method	NQ		PopQA		HotpotQA		2Wiki		MuSiQue		MultiHop-RAG		Average	
		EM	F1	EM	F1	EM	F1	EM	F1	EM	F1	EM	F1	EM	F1
Traditional RAG	BM25	45.4	56.76	40.9	49.01	43.8	54.07	34.7	36.90	13.0	17.96	2.5	3.93	23.55	36.44
	DPR	46.0	59.09	47.2	55.89	52.0	63.10	39.9	43.13	18.7	24.03	41.3	42.72	40.85	47.99
	BM25 + DPR + Q3R	48.2	60.58	<u>47.2</u>	56.01	58.4	70.98	41.6	45.43	28.3	34.81	52.9	53.49	46.10	53.55
	DPR + IRCOT + Q3R	<u>50.1</u>	62.71	47.3	56.98	59.1	71.00	48.9	53.44	29.7	37.36	41.7	42.28	46.13	53.96
Graph-RAG	GraphRAG	36.0	50.85	32.7	42.13	54.9	66.30	52.2	56.62	25.8	30.70	37.3	39.08	39.82	47.61
	HippoRAG 2	47.4	62.42	43.4	55.90	61.9	75.40	64.8	71.35	37.5	<u>48.00</u>	17.7	19.36	45.45	55.41
	HippoRAG 2 + Q3R	45.7	60.99	43.3	56.05	58.7	71.52	60.1	66.00	32.9	43.27	17.6	19.37	43.05	52.87
Tree-RAG	RAPTOR + Q3R	43.5	55.40	19.8	25.76	43.1	52.54	19.1	22.16	16.6	21.10	42.5	43.20	30.77	36.69
	Ψ -RAG $\boxplus$	50.0	<u>63.45</u>	46.7	<u>56.74</u>	<u>62.1</u>	74.59	69.1	<u>76.73</u>	38.7	48.90	55.3	56.23	53.65	62.77
	Ψ -RAG $\boxplus$	50.6	63.58	46.1	55.88	62.2	<u>74.85</u>	<u>69.0</u>	76.94	<u>37.9</u>	47.83	<u>54.1</u>	<u>55.18</u>	<u>53.32</u>	<u>62.38</u>

Table 3. Retrieval results for single- and multi-hop QA in Recall@2 (R@2%) and Recall@5 (R@5%). Best results are in **bold**. Second-bests are underlined.

Type	Method	NQ		PopQA		HotpotQA		2Wiki		MuSiQue		Average	
		R@2	R@5	R@2	R@5	R@2	R@5	R@2	R@5	R@2	R@5	R@2	R@5
Traditional RAG	BM25	28.20	56.03	23.90	35.70	57.25	74.80	54.70	64.98	28.78	35.40	38.57	53.38
	DPR	27.13	69.51	20.85	49.70	35.25	86.75	32.30	71.03	33.11	48.08	29.73	65.01
	BM25 + DPR + Q3R	46.29	<u>76.63</u>	42.90	50.55	85.70	93.80	68.95	74.52	50.92	67.35	58.95	72.57
	DPR + IRCOT + Q3R	<u>46.16</u>	76.10	42.75	50.45	84.50	92.40	69.20	74.08	50.90	66.02	58.70	71.81
Graph-RAG	HippoRAG 2	45.32	77.11	44.30	<u>50.75</u>	84.05	94.40	75.40	89.92	55.67	73.77	60.95	77.19
	HippoRAG 2 + Q3R	42.09	73.68	46.60	50.45	74.70	90.65	70.00	80.73	49.02	64.64	56.48	72.03
Tree-RAG	RAPTOR + Q3R	41.56	68.48	21.20	23.80	67.10	72.25	40.93	44.57	33.29	42.26	40.82	50.27
	Ψ -RAG $\boxplus$	46.08	75.72	<u>43.35</u>	51.30	88.15	96.00	81.35	96.13	57.21	75.77	63.23	78.98
	Ψ -RAG $\boxplus$	45.88	75.13	42.60	50.55	<u>88.10</u>	<u>95.80</u>	<u>81.23</u>	<u>95.85</u>	<u>55.69</u>	75.08	<u>62.70</u>	<u>78.48</u>

Ψ -RAG is equipped with Qwen3-Reranker-8B (Q3R) to fuse retrieval results from sparse and dense retrievers. For a fair comparison, most baselines are also equipped with Q3R. Detailed configurations and instruction prompts are provided in Appendix C.

5.2. Single- and Multi-Hop QA

We use the symbols “ $\boxplus$ ” and “ $\boxplus$ ” to denote Ψ -RAG with summative and keyword abstract, respectively. Retrieval and QA results on single-hop and multi-hop datasets are presented in Table 2 and 3, yielding several key observations.

(1) **Traditional baselines excel at single-hop QA.** For factual questions with little context such as NQ and PopQA, DPR and its hybrid combination with BM25 achieve recall and accuracy competitive with HippoRAG 2 and Ψ -RAG. In such cases, answers often lie in documents with frequent query-related entities or strong semantic correlation.

(2) **Multi-hop QA is the domain of iterative agentic RAG and structured RAG.** Both IRCOT and HippoRAG 2 outperform the hybrid retriever BM25 + DPR on 3 out of 4 multi-hop datasets, with the highest F1 gains exceeding 20%. This underscores the necessity of modeling implicit entity relationships for multi-hop reasoning.

(3) **RAPTOR’s effectiveness diminishes at corpus scale.**

On 5 out of 6 datasets, RAPTOR + Q3R underperforms DPR without a reranker, showing degradations exceeding 25% in Recall@5 and 30% in F1 on PopQA. This highlights a key limitation of traditional Tree-RAG: as the index space expands, performance on factual QA is severely hampered by coarse-grained abstracts.

(4) **Ψ -RAG achieves strong performance across both task types.** Compared to RAPTOR, Ψ -RAG achieves average gains of 25.9% in F1 and 23.7% in Recall@5, irrespective of the abstraction type. Notably, Ψ -RAG surpasses RAPTOR by up to 54.78% in F1 on 2Wiki. To our knowledge, Ψ -RAG is the first Tree-RAG to surpass advanced Graph-RAG, outperforming HippoRAG 2 by 7.4% and HippoRAG 2 + Q3R by 9.9% in average F1. We also find that the summative abstract slightly outperforms the keyword abstract. This indicates that logical relationships between entities can effectively compensate for abstract coarseness.

5.3. Narrative QA and Summarization

Table 4 shows the performance of narrative QA and summarization. We derive the following observations:

(1) **Traditional baselines struggle to capture causal relationships and high-level semantics.** Unlike in token-level QA, BM25 and DPR achieve the lowest performance in most cases. This indicates that simple keyword matching

Table 4. Narrative QA results in F1 (%) and ROUGE-L (R-L%), and summarization results in ROUGE-L (R-L%). Best results are in **bold**. Second-bests are underlined.

Method	NarrativeQA		∞ -LongBook		QMSum	WCEP
	F1	R-L	F1	R-L	R-L	R-L
BM25	13.95	14.07	14.87	15.54	15.63	16.08
DPR	17.78	17.87	21.51	23.60	13.43	8.14
BM25 + DPR + Q3R	20.30	20.37	24.46	24.87	13.42	8.14
GoR + Q3R	–	–	–	–	15.98	13.98
RAPTOR + Q3R	21.36	21.61	13.90	15.85	16.53	18.33
Ψ -RAG 	27.84	28.43	<u>25.52</u>	<u>27.11</u>	16.87	20.28
Ψ -RAG 	27.66	<u>28.35</u>	28.44	28.36	<u>16.65</u>	<u>18.81</u>

and vector similarity are insufficient for passage-level and document-level user demands. On the contrary, structured indexes of RAPTOR and GoR empower the agent with high-level information and thus higher performance on passage- and document-level tasks.

(2) **Ψ -RAG excels in both narrative and summarization tasks.** It consistently outperforms all baseline methods across both settings, surpassing RAPTOR by an average of 9.7% in F1 on NarrativeQA and ∞ -LongBook. It also surpasses GoR by 3.2% in average ROUGE-L.

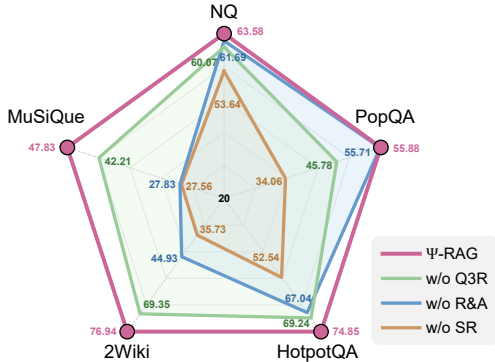


Figure 6. Ablation studies in F1 (%).

5.4. Ablation Studies

Figure 6 presents the QA performance of Ψ -RAG with different components ablated: the R&A agent, sparse retrieval (SR), and reranker (Q3R). We use RRF instead of the reranker in the “w/o Q3R” setting. We have some key observations:

(1) **Ψ -RAG achieves peak performance when all components are combined together.** Our R&A agent consistently improves performance on multi-hop QA tasks with an average gain of $\sim 20\%$. The contribution of sparse retrieval is also significant, especially on datasets like PopQA (+21.82%) and 2Wiki (+41.21%) with large amounts of short factual questions related to specific names. This validates its effectiveness in mitigating the coarse abstraction of the tree index. Furthermore, despite additional computational and storage costs, a large reranker integrates retrievers

Table 5. Query reorganization (QR) ablation in F1 (%). Best results are in **bold**.

	HotpotQA	2Wiki	MuSiQue	MultiHop-RAG
Ψ -RAG	74.85	76.94	47.83	55.18
w/o QR	74.17 ($\downarrow$ 0.68)	76.71 ($\downarrow$ 0.23)	45.82 ($\downarrow$ 2.01)	55.03 ($\downarrow$ 0.15)

more effectively than the non-parametric RRF, but it does not provide a core performance contribution in Ψ -RAG.

(2) **R&A agent is also beneficial for single-hop QA.** For example, it brings a 1.89% F1 gain on NQ. This benefit stems from its query reorganization mechanism, which improves retrieval for queries with insufficient context.

Query reorganization. Table 5 evaluates a counterpart of Ψ -RAG without query reorganization, that is, the R&A agent is no longer encouraged to reorganize and enrich their generated queries at each retrieval attempt. This leads to performance degradation across multi-hop datasets, with a 2% F1 drop on MuSiQue. Therefore, the thematic keywords and high-level context introduced by query reorganization contribute to accurate hybrid retrieval of Ψ -RAG.

Table 6. Experiment results of Ψ -RAG with different open-source backbones. Replaced components are shaded.

Embedding	Abstraction	R&A	Reranker	HotpotQA		2Wiki		MuSiQue	
				R@5	F1	R@5	F1	R@5	F1
Q3-8B	L3-70B	L3-70B	Q3R	95.80	74.85	95.85	76.94	75.08	47.83
Q3-0.6B	L3-70B	L3-70B	Q3R	94.40	74.48	95.55	75.94	73.42	46.57
Q3-8B	L3-1B	L3-70B	Q3R	94.45	73.23	95.77	76.78	73.14	47.00
Q3-8B	L3-8B	L3-70B	Q3R	93.65	73.72	95.83	76.30	74.29	47.46
Q3-8B	L3-70B	G3-27B	Q3R	95.00	73.65	94.05	71.78	71.80	49.30
Q3-8B	L3-70B	L3-70B	BGE	95.10	74.32	95.80	76.85	72.46	48.40

Table 7. Experiment results of Ψ -RAG with different closed-source R&A backbones.

R&A	HotpotQA		MuSiQue	
	R@5	F1	R@5	F1
 GPT-5-mini	94.40	78.10	72.67	49.48
 Gemini-2.5-Flash	96.00	77.18	77.06	51.86

5.5. Backbone Flexibility

Ψ -RAG can flexibly accommodate different backbone language models for its embedding model, abstraction agent, R&A agent, and reranker. We select various open-source models, including (1) Qwen3-Embedding-0.6B (Q3-0.6B) (Zhang et al., 2025b) as the embedding model; (2) Llama-3.2-1B (L3-1B) and Llama-3.1-8B (L3-8B) (Dubey et al., 2024) as the abstraction agent; (3) Gemma-3-27B (G3-27B) (Kamath et al., 2025) as the R&A agent; and (4) BGE-Reranker-Large (BGE) (Xiao et al., 2024) as the reranker. The results are shown in Table 6. We observe that these models, especially L3-1B/8B and BGE, serve as effective lightweight alternatives, as they achieve comparable or even superior performance with lower costs.

Table 8. Time costs of different structured RAG. All experiments follow the settings in Appendix C. The embedding time of Q3-8B for HotpotQA and MuSiQue is 277.3s and 309.0s, respectively. L3-70B is used by default for abstraction of both RAPTOR and Ψ -RAG. “ $\uparrow$ ” and “ $\downarrow$ ” indicate a distinguishable higher and lower efficiency compared to RAPTOR + Q3R, respectively.

Method	Indexing time		Average single retrieval time		Average QA time	
	HotpotQA	MuSiQue	HotpotQA	MuSiQue	HotpotQA	MuSiQue
HippoRAG 2	OpenIE: 105,360s Graph indexing: 32,187s Total: 137,547s $\downarrow$	OpenIE: 128,192s Graph indexing: 20,668s Total: 148,860s $\downarrow$	Graph retrieval: 10.44s $\downarrow$	Graph retrieval: 8.94s $\downarrow$	19.24s $\downarrow$	17.56s $\downarrow$
RAPTOR + Q3R	Tree indexing: 1,123s Abstraction: 5,509s Total: 6,632s	Tree indexing: 1,684s Abstraction: 7,570s Total: 9,254s	Tree retrieval: 0.22s Reranking: 0.98s Total: 1.20s	Tree retrieval: 0.22s Reranking: 2.17s Total: 2.39s	Single retrieval: 8.03s	Single retrieval: 12.71s
Ψ -RAG	Tree indexing: 124s $\uparrow$ Abstraction: 10,425s $\downarrow$ Total: 10,549s $\downarrow$	Tree indexing: 258s $\uparrow$ Abstraction: 11,899s $\downarrow$ Total: 12,157s $\downarrow$	Tree retrieval: 0.19s Sparse retrieval: < 0.01s Reranking: 0.79s Total: 0.98s $\uparrow$	Tree retrieval: 0.19s Sparse retrieval: < 0.01s Reranking: 1.07s $\uparrow$ Total: 1.26s $\uparrow$	Agentic retrieval: 8.49s	Agentic retrieval: 19.66s $\downarrow$
Ψ -RAG (Abstraction: L3-8B; R&A Agent: G3-27B; Reranker: BGE)	Tree indexing: 128s $\uparrow$ Abstraction: 2,058s $\uparrow$ Total: 2,186s $\uparrow$	Tree indexing: 265s $\uparrow$ Abstraction: 2,222s $\uparrow$ Total: 2,487s $\uparrow$	Tree retrieval: 0.19s Sparse retrieval: < 0.01s Reranking: 0.56s $\uparrow$ Total: 0.75s $\uparrow$	Tree retrieval: 0.19s Sparse retrieval: < 0.01s Reranking: 0.37s $\uparrow$ Total: 0.56s $\uparrow$	Agentic retrieval: 5.45s $\uparrow$	Agentic retrieval: 7.17s $\uparrow$

We also employ two popular closed-source LLMs: GPT-5-Mini (OpenAI, 2025) and Gemini-2.5-Flash (Comanici et al., 2025), as the R&A agent in Ψ -RAG. The results are presented in Table 7. Even with a much smaller size than L3-70B, both models yield certain improvements in F1 score. For instance, employing Gemini-2.5-Flash as the R&A agent achieves a notably high F1 score of 51.86%.

5.6. Efficiency Analysis

As Ψ -RAG is based on AHC, the computation, sorting, and storage of the similarity matrix introduce time and space complexities of $O(n^2 \log n)$ and $O(n^2)$, respectively³. Similarly, the iterative merging and collapse process takes $O(n^2)$ time. However, the highly optimized matrix multiplication and sorting techniques make our proposed framework still efficient for a domain-specific index.

Table 8 compares the time efficiency of Ψ -RAG with other baseline methods. Our abstract tree indexing is very fast, taking only 258 seconds on MuSiQue with 1.3 million tokens (the similarity ranking process only takes ~ 30 s), which is $6.5\times$ faster than RAPTOR and considerably more efficient than HippoRAG 2 on domain-specific corpora⁴. However, as Ψ -RAG includes more abstract nodes, it takes more time for abstraction. For retrieval efficiency, the top-down search algorithm inherently has $O(\log n)$ complexity, resulting in lower average retrieval latency compared to HippoRAG 2. While Ψ -RAG’s iterative agentic retrieval process increases the latency, it is proportional to the complexity of the user demand (one additional retrieval attempt opportunity only brings an increment of 0.46s compared to RAPTOR on

HotpotQA), so we believe that it is justified in quality improvement for complex queries.

Considering the limitations of similarity ranking when applied to industrial-scale corpora with 10M+ and even 100M+ tokens, we further propose two efficient tree-building extensions for Ψ -RAG: **Bucketing** and **Approximate Nearest Neighbor Search with Hierarchical Navigable Small World (HNSW)** (Malkov & Yashunin, 2018). See Appendix E for details.

More experiment results. Appendix D provides more extensive results, including case studies, failure studies, and more analyses towards the effectiveness of Ψ -RAG components.

6. Conclusion

We propose Ψ -RAG, a hierarchical Tree-RAG framework supporting corpus-level indexing and cross-document multi-hop user questions. By a hierarchical clustering-based abstract tree index and a multi-granular agentic retriever, Ψ -RAG addresses key limitations of k -means-type tree indexes such as poor distribution adaptability, structural isolation, and coarse abstraction. Extensive experimental results demonstrate that Ψ -RAG achieves substantial gains across tasks of diverse granularities, promoting the development of tree-RAG in the rapidly evolving era of LLMs.

Acknowledgements

This work was partially supported by the Guangdong Provincial Natural Science Foundation General Program (Grant No. 2026A1515012118).

We thank Linxiao Cao for his assistance in the comparative experiment. We also thank the anonymous reviewers, the Associate Chair, the Program Chair, etc. for their constructive feedback and recognition of this work.

³We omit the vector dimension d as $d \ll n$ in the real-world scenario.

⁴Existing Graph-RAG papers tend not to include OpenIE extraction in the calculation of indexing time. This may lead to a significant inconsistency between the reported time and the actual running time.

Impact Statement

This work focuses on retrieval-augmented generation (RAG) for large language models (LLMs) with a structured index. While this work may have broad applications in knowledge-intensive natural language processing tasks, we do not identify any societal impacts that extend beyond those common to LLMs and retrieval systems.

References

- Arivazhagan, M. G., Liu, L., Qi, P., Chen, X., Wang, W. Y., and Huang, Z. Hybrid hierarchical retrieval for open-domain question answering. In *Findings of the Association for Computational Linguistics: ACL 2023*, pp. 10680–10689, 2023.
- Asai, A., Wu, Z., Wang, Y., Sil, A., and Hajishirzi, H. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *Proceedings of the 12th International Conference on Learning Representations*, 2024.
- Assran, M., Balestrieri, R., Duval, Q., Bordes, F., Misra, I., Bojanowski, P., Vincent, P., Rabbat, M., and Ballas, N. The hidden uniform cluster prior in self-supervised learning. In *Proceedings of the 11th International Conference on Learning Representations*, 2023.
- Banko, M., Cafarella, M. J., Soderland, S., Broadhead, M., and Etzioni, O. Open information extraction from the web. In *Proceedings of the 20th International Joint Conference on Artificial Intelligence*, pp. 2670–2676, 2007.
- Bentley, J. L. Multidimensional binary search trees used for associative searching. *Communications of the ACM*, 18(9):509–517, 1975.
- Chami, I., Gu, A., Chatziafratis, V., and Ré, C. From trees to continuous embeddings and back: Hyperbolic hierarchical clustering. In *Proceedings of the 34th International Conference on Neural Information Processing Systems*, volume 33, pp. 15065–15076, 2020.
- Charikar, M. and Chatziafratis, V. Approximate hierarchical clustering via sparsest cut and spreading metrics. In *Proceedings of the 38th Annual ACM-SIAM Symposium on Discrete Algorithms*, pp. 841–854. SIAM, 2017.
- Chen, B., Guo, Z., Yang, Z., Chen, Y., Chen, J., Liu, Z., Shi, C., and Yang, C. PathRAG: Pruning graph-based retrieval augmented generation with relational paths. In *Proceedings of the 40th AAAI Conference on Artificial Intelligence*, pp. 30183–30191, 2026.
- Chen, T., Wang, H., Chen, S., Yu, W., Ma, K., Zhao, X., Zhang, H., and Yu, D. Dense X Retrieval: What retrieval granularity should we use? In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 15159–15177, 2024.
- Comanici, G., Bieber, E., Schaekermann, M., Pasupat, I., Sachdeva, N., Dhillon, I., Blistein, M., Ram, O., Zhang, D., Rosen, E., et al. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261*, 2025.
- Cormack, G. V., Clarke, C. L., and Buettcher, S. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval*, pp. 758–759, 2009.
- Dasgupta, S. A cost function for similarity-based hierarchical clustering. In *Proceedings of the forty-eighth annual ACM symposium on Theory of Computing*, pp. 118–127, 2016.
- Douze, M., Guzhva, A., Deng, C., Johnson, J., Szilvasy, G., Mazaré, P.-E., Lomeli, M., Hosseini, L., and Jégou, H. The Faiss library. *IEEE Transactions on Big Data*, 2025.
- Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Yang, A., Fan, A., et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., Metropolitansky, D., Ness, R. O., and Larson, J. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.
- Fan, W., Ding, Y., Ning, L., Wang, S., Li, H., Yin, D., Chua, T.-S., and Li, Q. A survey on RAG meeting LLMs: Towards retrieval-augmented large language models. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 6491–6501, 2024.
- Fatehkia, M., Lucas, J. K., and Chawla, S. T-RAG: lessons from the LLM trenches. *arXiv preprint arXiv:2402.07483*, 2024.
- Florek, K., Łukaszewicz, J., Perkal, J., Steinhaus, H., and Zubrzycki, S. Sur la liaison et la division des points d’un ensemble fini. In *Colloquium Mathematicum*, volume 2, pp. 282–285, 1951.
- Gao, L., Dai, Z., Chen, T., Fan, Z., Van Durme, B., and Callan, J. Complement lexical retrieval model with semantic residual embeddings. In *43rd European Conference on Information Retrieval*, pp. 146–160. Springer, 2021.

- Ghalandari, D. G., Hokamp, C., Glover, J., Ifrim, G., et al. A large-scale multi-document summarization dataset from the Wikipedia current events portal. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 1302–1308, 2020.
- Guo, Z., Xia, L., Yu, Y., Ao, T., and Huang, C. LightRAG: Simple and fast retrieval-augmented generation. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pp. 10746–10761, 2025.
- Gupta, N., Chang, W.-C., Bui, N., Hsieh, C.-J., and Dhillon, I. S. LLM-guided hierarchical retrieval. *arXiv preprint arXiv:2510.13217*, 2025.
- Ho, X., Nguyen, A.-K. D., Sugawara, S., and Aizawa, A. Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps. In *Proceedings of the 28th International Conference on Computational Linguistics*, pp. 6609–6625, 2020.
- Huang, H., Huang, Y., Yang, J., Pan, Z., Chen, Y., Ma, K., Chen, H., and Cheng, J. Retrieval-augmented generation with hierarchical knowledge. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pp. 6044–6060, 2025.
- Izacard, G., Caron, M., Hosseini, L., Riedel, S., Bojanowski, P., Joulin, A., and Grave, E. Unsupervised dense information retrieval with contrastive learning. *Transactions on Machine Learning Research*, 2022.
- Jardine, N. and van Rijsbergen, C. J. The use of hierarchic clustering in information retrieval. *Information Storage and Retrieval*, 7(5):217–240, 1971.
- Jiménez Gutiérrez, B., Shu, Y., Gu, Y., Yasunaga, M., and Su, Y. HippoRAG: Neurobiologically inspired long-term memory for large language models. In *Proceedings of the 38th Conference in Neural Information Processing Systems*, pp. 59532–59569, 2024.
- Jiménez Gutiérrez, B., Shu, Y., Qi, W., Zhou, S., and Su, Y. From RAG to memory: Non-parametric continual learning for large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, pp. 21497–21515, 2025.
- Jin, B., Yoon, J., Han, J., and Arik, S. O. Long-context LLMs meet RAG: Overcoming challenges for long inputs in RAG. In *Proceedings of the 13th International Conference on Learning Representations*, 2025a.
- Jin, J., Li, X., Dong, G., Zhang, Y., Zhu, Y., Wu, Y., Li, Z., Qi, Y., and Dou, Z. Hierarchical document refinement for long-context retrieval-augmented generation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 3502–3520, 2025b.
- Jin, J., Zhu, Y., Dou, Z., Dong, G., Yang, X., Zhang, C., Zhao, T., Yang, Z., and Wen, J.-R. FlashRAG: A modular toolkit for efficient retrieval-augmented generation research. In *Companion Proceedings of the ACM on Web Conference*, pp. 737–740, 2025c.
- Jones, C. H. Generalized hockey stick identities and n-dimensional blockwalking. *The Fibonacci Quarterly*, 34(3):280–288, 1996.
- Kamath, A., Ferret, J., Pathak, S., Vieillard, N., Merhej, R., Perrin, S., Matejovicova, T., Ramé, A., Rivière, M., et al. Gemma 3 technical report. *arXiv preprint arXiv:2503.19786*, 2025.
- Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., and Yih, W.-t. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 6769–6781, 2020.
- Khattab, O. and Zaharia, M. ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*, pp. 39–48, 2020.
- Kočiský, T., Schwarz, J., Blunsom, P., Dyer, C., Hermann, K. M., Melis, G., and Grefenstette, E. The NarrativeQA reading comprehension challenge. *Transactions of the Association for Computational Linguistics*, 6:317–328, 2018.
- Kwiatkowski, T., Palomaki, J., Redfield, O., Collins, M., Parikh, A., Alberti, C., Epstein, D., Polosukhin, I., Devlin, J., Lee, K., et al. Natural Questions: a benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466, 2019.
- Lee, C., Roy, R., Xu, M., Raiman, J., Shoeybi, M., Catanzaro, B., and Ping, W. NV-Embed: Improved techniques for training LLMs as generalist embedding models. In *Proceedings of the 13th International Conference on Learning Representations*, 2025.
- Lee, D., Hwang, S.-w., Lee, K., Choi, S., and Park, S. On complementarity objectives for hybrid retrieval. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 13357–13368, 2023.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Proceedings of the 34th Conference on Neural Information Processing Systems*, pp. 9459–9474, 2020.

- Li, M., Miao, S., and Li, P. Simple is effective: The roles of graphs and large language models in knowledge-graph-based retrieval-augmented generation. In *Proceedings of the 13th International Conference on Learning Representations*, 2025a.
- Li, X., Bai, Y., Jin, B., Zhu, F., Pan, L., and Cao, Y. Long context vs. RAG: Strategies for processing long documents in LLMs. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 4110–4113, 2025b.
- Li, X., Dong, G., Jin, J., Zhang, Y., Zhou, Y., Zhu, Y., Zhang, P., and Dou, Z. Search-o1: Agentic search-enhanced large reasoning models. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 5420–5438, 2025c.
- Li, Y., Zhang, W., Yang, Y., Huang, W.-C., Wu, Y., Luo, J., Bei, Y., Zou, H. P., Luo, X., Zhao, Y., et al. Towards agentic RAG with deep reasoning: A survey of RAG-reasoning systems in LLMs. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025d.
- Liang, J., Bai, L., Dang, C., and Cao, F. The K-means-type algorithms versus imbalanced data distributions. *IEEE Transactions on Fuzzy Systems*, 20(4):728–745, 2012.
- Liu, J. LlamaIndex, 2022. URL https://github.com/jerryjliu/llama_index.
- Liu, Y., Hashimoto, K., Zhou, Y., Yavuz, S., Xiong, C., and Yu, P. S. Dense hierarchical retrieval for open-domain question answering. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, pp. 188–200, 2021.
- Lù, X. H. BM25S: Orders of magnitude faster lexical search via eager sparse scoring. *arXiv preprint arXiv:2407.03618*, 2024.
- Malkov, Y. A. and Yashunin, D. A. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2018.
- Mallen, A., Asai, A., Zhong, V., Das, R., Khashabi, D., and Hajishirzi, H. When not to trust language models: Investigating effectiveness of parametric and non-parametric memories. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 9802–9822, 2023.
- Moseley, B. and Wang, J. R. Approximation bounds for hierarchical clustering: average linkage, bisecting K-means, and local search. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, pp. 3097–3106, 2017.
- Nair, I., Garimella, A., Srinivasan, B. V., Modani, N., Chhaya, N., Karanam, S., and Shekhar, S. A neural CRF-based hierarchical approach for linear text segmentation. In *Findings of the Association for Computational Linguistics: EACL 2023*, pp. 883–893, 2023.
- Omohundro, S. M. *Five balltree construction algorithms*. International Computer Science Institute Berkeley, 1989.
- OpenAI. Introducing GPT-5. 2025. URL <https://openai.com/index/introducing-gpt-5/>.
- Peng, B., Zhu, Y., Liu, Y., Bo, X., Shi, H., Hong, C., Zhang, Y., and Tang, S. Graph retrieval-augmented generation: A survey. *ACM Transactions on Information Systems*, 2025.
- Rezazadeh, A., Li, Z., Wei, W., and Bao, Y. From isolated conversations to hierarchical schemas: Dynamic tree memory representation for LLMs. In *Proceedings of the 13th International Conference on Learning Representations*, 2025.
- Robertson, S., Zaragoza, H., et al. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends® in Information Retrieval*, 3(4):333–389, 2009.
- Sarathi, P., Abdullah, S., Tuli, A., Khanna, S., Goldie, A., and Manning, C. D. RAPTOR: Recursive abstractive processing for tree-organized retrieval. In *Proceedings of the 12th International Conference on Learning Representations*, 2024.
- Sokal, R. R. and Michener, C. D. A statistical method for evaluating systematic relationships. 1958.
- Sparck Jones, K. A statistical interpretation of term specificity and its application in retrieval. *Journal of documentation*, 28(1):11–21, 1972.
- Tang, Y. and Yang, Y. MultiHop-RAG: Benchmarking retrieval-augmented generation for multi-hop queries. In *First Conference on Language Modeling*, 2024.
- Tao, W., Xing, X., Chen, Y., Huang, L., and Xu, X. TreeRAG: Unleashing the power of hierarchical storage for enhanced knowledge retrieval in long documents. In *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 356–371, 2025.
- Trivedi, H., Balasubramanian, N., Khot, T., and Sabharwal, A. MuSiQue: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 10:539–554, 2022.

- Trivedi, H., Balasubramanian, N., Khot, T., and Sabharwal, A. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: long papers)*, pp. 10014–10037, 2023.
- Wang, S., Fang, Y., Zhou, Y., Liu, X., and Ma, Y. ArchRAG: Attributed community-based hierarchical retrieval-augmented generation. In *Proceedings of the 40th AAAI Conference on Artificial Intelligence*, pp. 15868–15876, 2026.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q. V., Zhou, D., et al. Chain-of-thought prompting elicits reasoning in large language models. In *Proceedings of the 36th Conference in Neural Information Processing Systems*, volume 35, pp. 24824–24837, 2022.
- Wu, Y., Wang, Y., Ye, Z., Du, T., Jegelka, S., and Wang, Y. When more is less: Understanding chain-of-thought length in LLMs. In *ICLR 2025 Workshop on Reasoning and Planning for Large Language Models*, 2025.
- Xiao, S., Liu, Z., Zhang, P., Muennighoff, N., Lian, D., and Nie, J.-Y. C-Pack: Packaged resources to advance general Chinese embedding. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 641–649, 2024.
- Xiao, Y., Dong, J., Zhou, C., Dong, S., Zhang, Q.-w., Yin, D., Sun, X., and Huang, X. GraphRAG-Bench: Challenging domain-specific reasoning for evaluating graph retrieval-augmented generation. *arXiv preprint arXiv:2506.02404*, 2025.
- Xiong, H., Wu, J., and Chen, J. K-means clustering versus validation measures: a data distribution perspective. In *Proceedings of the 12th ACM SIGKDD international conference on Knowledge discovery and data mining*, pp. 779–784, 2006.
- Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, W., Salakhutdinov, R., and Manning, C. D. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 conference on empirical methods in natural language processing*, pp. 2369–2380, 2018.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. R., and Cao, Y. ReAct: Synergizing reasoning and acting in language models. In *Proceedings of the 11th International Conference on Learning Representations*, 2022.
- Zhang, H., Feng, T., and You, J. Graph of records: Boosting retrieval augmented generation for long-context summarization with graphs. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 23780–23799, 2025a.
- Zhang, X., Chen, Y., Hu, S., Xu, Z., Chen, J., Hao, M., Han, X., Thai, Z., Wang, S., Liu, Z., et al. ∞ Bench: Extending long context evaluation beyond 100k tokens. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 15262–15277, 2024a.
- Zhang, X., Wang, M., Yang, X., Wang, D., Feng, S., and Zhang, Y. Hierarchical retrieval-augmented generation model with rethink for multi-hop question answering. *arXiv preprint arXiv:2408.11875*, 2024b.
- Zhang, Y., Li, M., Long, D., Zhang, X., Lin, H., Yang, B., Xie, P., Yang, A., Liu, D., Lin, J., et al. Qwen3 Embedding: Advancing text embedding and reranking through foundation models. *arXiv preprint arXiv:2506.05176*, 2025b.
- Zhao, Y., Zhu, J., Guo, Y., He, K., and Li, X. E²GraphRAG: Streamlining graph-based RAG for high efficiency and effectiveness. *arXiv preprint arXiv:2505.24226*, 2025.
- Zhong, M., Yin, D., Yu, T., Zaidi, A., Mutuma, M., Jha, R., Hassan, A., Celikyilmaz, A., Liu, Y., Qiu, X., et al. QM-Sum: A new benchmark for query-based multi-domain meeting summarization. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 5905–5921, 2021.

Appendix Contents

A Algorithms of Ψ-RAG	15
B Theoretical Proofs	16
B.1 Proof of Theorem 4.1	16
B.2 Proof of Theorem 4.3	17
B.3 Proof of Theorem 4.4	20
C Detailed Experimental Setup	22
C.1 Dataset and Index Statistics	22
C.2 Configurations	22
C.3 Instruction Prompts	23
D Supplemental Experiments	25
D.1 Case Study	25
D.2 Analyses on Hierarchical Abstract Tree	27
D.3 Analyses on Multi-granular Agentic Retriever	27
D.4 Failure Analysis	28
E Real-World Extensions	29
E.1 Approximate Nearest Neighbor Search for Efficient Indexing	29
E.2 Query Hop Discriminator	29
F Related Work	30
G Discussion and Future Work	30

A. Algorithms of Ψ -RAG

Algorithm 1 shows the algorithm of Abstract Tree Indexing, while Algorithm 2 and Algorithm 3 show the algorithm of Multi-granular Agentic Retrieval.

Algorithm 1 Abstract Tree Indexing

Input: pairwise similarity matrix $\mathbf{S}$
 Descending sort $s(i, j)$ for all $1 \leq i < j \leq n$
 Initialize $\mathcal{T} = (\mathcal{V} = \emptyset, \mathcal{E} = \emptyset)$
for $rank(u, v) = 1$ **to** $n - 1$, $(u, v) \in \mathbf{S}$ **do**
 if $p(u) = p(v) = \emptyset$ **then**
 Add node a in $\mathcal{V}$ and set $c(a) = \{u, v\}$
 else if $root(u) \neq root(v)$ **and** $\delta(u) = \delta(v)$ **then**
 Add node a in $\mathcal{V}$ and set $c(a) = \{root(u), root(v)\}$
 else if $root(u) \neq root(v)$ **and** $\delta(u) > \delta(v)$ **then**
 $c(p^{\delta(v)+1}(u)) \leftarrow c(p^{\delta(v)+1}(u)) \cup \{root(v)\}$
 else if $root(u) \neq root(v)$ **and** $\delta(u) < \delta(v)$ **then**
 $c(p^{\delta(u)+1}(v)) \leftarrow c(p^{\delta(u)+1}(v)) \cup \{root(u)\}$
 end if
end for
 Tree rebalancing by splitting overload nodes
 Generate abstract text for every abstract node
Output: abstract tree $\mathcal{T}$

Algorithm 2 Tree Retrieval $r(q, \mathcal{T})$

Input: query q , tree index $\mathcal{T}$, number of retrieved chunks k , maximum retrieval step $i_{\max}$
 Initialize candidate nodes $\mathcal{V}_0 = \{root(\mathcal{T})\}$
for $l = 0$ **to** $\delta(\mathcal{T})$ **do**
 if $l = \delta(\mathcal{T})$ **and** $|\{u | \delta(u) = l\}| \leq k$ **then**
 $\mathcal{D}^* = \ell(\mathcal{T})$
 break
 else if $|\{u | \delta(u) = l\}| \leq k$ **then**
 $\mathcal{V}_{l+1} = \{u | \delta(u) = l + 1\}$
 continue
 end if
 {Top- k nearest neighbor search}
 $\mathcal{U}_l = topk(\mathcal{V}_l, q; k)$
 if $l = \delta(\mathcal{T})$ **then**
 $\mathcal{D}^* = \mathcal{U}_l$
 else
 $\mathcal{V}_{l+1} = \bigcup_{u \in \mathcal{U}_l} c(u)$
 end if
end for
Output: top- k chunks $\mathcal{D}^*$

Algorithm 3 Multi-granular Agentic Retrieval

Input: user query q , tree index $\mathcal{T}$, sparse index $\mathcal{I}$, LLM agent $f(\cdot; I(\cdot))$, number of retrieved chunks k
 {Hybrid retrieval}
 Initialize $\mathcal{D}_0^* = r(q, \mathcal{T}) \cup r(q, \mathcal{I})$
 Current retrieval step $i = 1$
repeat
 if $i = i_{\max}$ **then**
 Set $I(\mathcal{D}_i^*)$ such that $\langle \text{action} \rangle = \langle \text{answer} \rangle$
 end if
 Generate $a_i = (R_i, \langle \text{action} \rangle, \cdot)$ from (3)
 if $a_i = (R_i, \langle \text{retrieve} \rangle, q'_i)$ **then**
 $\mathcal{D}_i^* = r(q'_i, \mathcal{T}) \cup r(q'_i, \mathcal{I})$
 $i \leftarrow i + 1$
 end if
until $a_i = (R, \langle \text{answer} \rangle, y)$
Output: answer y

B. Theoretical Proofs

B.1. Proof of Theorem 4.1

Theorem 4.1 (Uniform effect of k -means (Liang et al., 2012)). *Given $\mathcal{L}$ in (4), we have*

$$\mathcal{L} = -A \sum_{1 \leq i < j \leq k} n_i n_j \|\mu_i - \mu_j\|^2 + B \quad (7)$$

where A and B are constants only relative to the data.

Proof. Let $d(c_i, c_j) = \sum_{x_i \in c_i, x_j \in c_j} \|x_i - x_j\|^2$. We first denote the sum of all pairwise distances of data samples $x \in \ell(\mathcal{T})$ within k clusters as

$$\mathcal{P}_k = \sum_{i=1}^n \sum_{j=1}^n \|x_i - x_j\|^2 = \sum_{i=1}^k d(c_i, c_i) + 2 \sum_{1 \leq i < j \leq k} d(c_i, c_j) \quad (8)$$

Then, we are going to prove

$$\mathcal{P}_k = \sum_{i=1}^k \frac{n}{n_i} d(c_i, c_i) + 2\mathcal{L}_k^{\leftrightarrow}, \quad (9)$$

where

$$\mathcal{L}_k^{\leftrightarrow} = \sum_{1 \leq i < j \leq k} n_i n_j \|\mu_i - \mu_j\|^2 \quad (10)$$

with mathematical induction.

For $k = 1$, we have $\mathcal{L}_1^{\leftrightarrow} = 0$ as condition $1 \leq i < j \leq k$ cannot be met. So $\mathcal{P}_1 = \sum_{x, y \in \ell(\mathcal{T})} \|x - y\|^2$ which meets (9).

For $k = 2$, we have

$$\mathcal{P}_2 = d(c_1, c_1) + d(c_2, c_2) + 2d(c_1, c_2) \quad (11)$$

$$= \sum_{1 \leq i < j \leq n_1} \|x_i - x_j\|^2 + \sum_{1 \leq i < j \leq n_2} \|x_i - x_j\|^2 + 2 \sum_{1 \leq i \leq n_1} \sum_{1 \leq j \leq n_2} \|x_i - x_j\|^2 \quad (12)$$

$$= 2(n_1 - 1) \sum_{i=1}^{n_1} \|x_i\|^2 - 4 \sum_{1 \leq i < j \leq n_1} x_i x_j + 2(n_1 - 2) \sum_{i=1}^{n_2} \|x_i\|^2 - 4 \sum_{1 \leq i < j \leq n_2} x_i x_j + 2n_2 \sum_{i=1}^{n_1} \|x_i\|^2 + 2n_1 \sum_{i=1}^{n_2} \|x_i\|^2 - 4 \sum_{1 \leq i \leq n_1} \sum_{1 \leq j \leq n_2} x_i x_j \quad (13)$$

$$= \left(\frac{2n(n_1 - 1)}{n_1} \sum_{i=1}^{n_1} \|x_i\|^2 - 4 \cdot \frac{n}{n_1} \sum_{1 \leq i < j \leq n_1} x_i x_j \right) + \left(\frac{2n(n_2 - 1)}{n_2} \sum_{i=1}^{n_2} \|x_i\|^2 - 4 \cdot \frac{n}{n_2} \sum_{1 \leq i < j \leq n_2} x_i x_j \right) + \left(\frac{2n_2}{n_1} \left(\sum_{i=1}^{n_1} \|x_i\|^2 + 2 \sum_{1 \leq i < j \leq n_1} x_i x_j \right) + \frac{2n_1}{n_2} \left(\sum_{i=1}^{n_2} \|x_i\|^2 + 2 \sum_{1 \leq i < j \leq n_2} x_i x_j \right) - 4 \sum_{1 \leq i \leq n_1} \sum_{1 \leq j \leq n_2} x_i x_j \right) \quad (14)$$

$$= \frac{n}{n_1} \sum_{1 \leq i < j \leq n_1} \|x_i - x_j\|^2 + \frac{n}{n_2} \sum_{1 \leq i < j \leq n_2} \|x_i - x_j\|^2 + 2n_1 n_2 \|\mu_1 - \mu_2\|^2 \quad (15)$$

$$= \frac{n}{n_1} d(c_1, c_1) + \frac{n}{n_2} d(c_2, c_2) + 2\mathcal{L}_2^{\leftrightarrow} \quad (16)$$

which meets (9).

Assume (9) holds with $k = K - 1$. Then for $k = K$, we have

$$\mathcal{P}_K = \mathcal{P}_{K-1}^{\hat{i}} + d(c_i, c_i) + 2 \sum_{1 \leq j \neq i \leq K} d(c_i, c_j) \quad (17)$$

where $\mathcal{P}_{K-1}^i$ is the sum of pairwise distances within $K-1$ clusters excluding cluster c_i for $i = 1, 2, \dots, K$.

We sum up (17) for every cluster c_i and get

$$K\mathcal{P}_K = \sum_{i=1}^K \mathcal{P}_{K-1}^i + \sum_{i=1}^K d(c_i, c_i) + 4 \sum_{1 \leq i < j \leq K} d(c_i, c_j) \quad (18)$$

With our assumption:

$$\mathcal{P}_{K-1}^i = \sum_{1 \leq j \neq i \leq K} \frac{n - n_i}{n_j} d(c_j, c_j) + 2 \sum_{1 \leq j < \kappa \neq i \leq K} n_j n_\kappa \|\mu_j - \mu_\kappa\|^2 \quad (19)$$

So

$$\sum_{i=1}^K \mathcal{P}_{K-1}^i = (K-2) \left(\sum_{i=1}^K \frac{n}{n_i} d(c_i, c_i) + 2 \sum_{1 \leq i < j \leq K} n_i n_j \|\mu_i - \mu_j\|^2 \right) + \sum_{i=1}^K d(c_i, c_i), \quad (20)$$

and from (8), we have

$$K\mathcal{P}_K = (K-2) \left(\sum_{i=1}^K \frac{n}{n_i} d(c_i, c_i) + 2\mathcal{L}_K^{\leftrightarrow} \right) + 2 \sum_{i=1}^K d(c_i, c_i) + 4 \sum_{1 \leq i < j \leq K} d(c_i, c_j) \quad (21)$$

$$= (K-2) \left(\sum_{i=1}^K \frac{n}{n_i} d(c_i, c_i) + 2\mathcal{L}_K^{\leftrightarrow} \right) + 2\mathcal{P}_K \quad (22)$$

$$\Rightarrow \mathcal{P}_K = \sum_{i=1}^K \frac{n}{n_i} d(c_i, c_i) + 2\mathcal{L}_K^{\leftrightarrow} \quad (23)$$

So (9) holds. With (9), we can now write $\mathcal{L}$ as

$$\mathcal{L} = \sum_{i=1}^k \sum_{x_i \in c_i} \|x_i - \mu_i\|^2 = \frac{1}{2} \sum_{i=1}^k \frac{d(c_i, c_i)}{n_i} \quad (24)$$

$$= \frac{1}{2n} \sum_{i=1}^k \left(\frac{n}{n_i} d(c_i, c_i) + 2\mathcal{L}_k^{\leftrightarrow} \right) - \frac{1}{n} \sum_{i=1}^k \mathcal{L}_k^{\leftrightarrow} \quad (25)$$

$$= -\frac{1}{n} \mathcal{L}_k^{\leftrightarrow} + \frac{\mathcal{P}_k}{2n}, \quad (26)$$

which completes the proof with $A = -\frac{1}{n}$ and $B = \frac{\mathcal{P}_k}{2n}$. $\square$

Remark. Theorem 4.1 proves that the objective of k -means is essentially maximizing $\mathcal{L}_k^{\leftrightarrow} = \sum_{1 \leq i < j \leq k} n_i n_j \|\mu_i - \mu_j\|^2$. If we assume $n_i n_j$ and $\|\mu_i - \mu_j\|^2$ is independent, the maximization of $n_i n_j$ leads to $n_1 = n_2 = \dots = n_k = n/k$.

B.2. Proof of Theorem 4.3

We first introduce necessary lemmas for Theorem 4.3 and Theorem 4.4.

Lemma B.1 (Modularity of Dasgupta's cost (Dasgupta, 2016)). *Let $\mathcal{T}_a$ be the subtree in $\mathcal{T}$ rooted at node a . If a different tree $\mathcal{T}'$ is constructed by replacing $\mathcal{T}_a$ with another subtree $\mathcal{T}'_a$ satisfying $\ell(\mathcal{T}'_a) = \ell(\mathcal{T}_a)$, then*

$$c(\mathcal{T}') = c(\mathcal{T}) - c(\mathcal{T}_a) + c(\mathcal{T}'_a) \quad (27)$$

Proof. For any leaf pair where one leaf is inside $\mathcal{T}'_a$ and the other is outside, the leaf count of its LCA is given by

$$c_{\text{cross}}(\mathcal{T}'_a, \mathcal{T}' \setminus \mathcal{T}'_a) = \sum_{u \in \mathcal{T}'_a, v \in \mathcal{T}' \setminus \mathcal{T}'_a} |\ell(\text{LCA}(u, v))| = n'_a + n_1 + n_2 + \dots \quad (28)$$

Here, the total number of leaves in the current subtree $n'_a = n_a$ remains unchanged regardless of the subtree structure. Therefore:

$$c(\mathcal{T}') = c(\mathcal{T}'_a) + c(\mathcal{T}' \setminus \mathcal{T}'_a) + c_{\text{cross}}(\mathcal{T}'_a, \mathcal{T}' \setminus \mathcal{T}'_a) \quad (29)$$

$$= c(\mathcal{T}'_a) + c(\mathcal{T} \setminus \mathcal{T}_a) + c_{\text{cross}}(\mathcal{T}_a, \mathcal{T} \setminus \mathcal{T}_a) \quad (30)$$

$$= c(\mathcal{T}'_a) + c(\mathcal{T}) - c(\mathcal{T}_a) \quad (31)$$

where $c(\mathcal{T} \setminus \mathcal{T}_a)$ denotes the sum of costs among the subtrees of $\mathcal{T}$ excluding $\mathcal{T}_a$. $\square$

Remark. The modularity of Dasgupta's cost allows us to, without loss of generality, decompose the problem of computing the global cost into analyzing cost differences within corresponding subtrees.

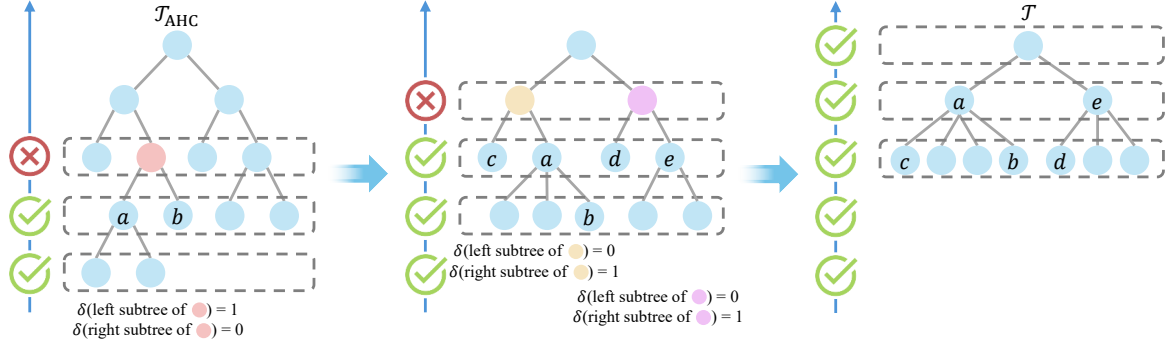


Figure 7. Illustration of $\psi(\cdot)$: bottom-up traversal and collapsing. Note that both $\mathcal{T}_{\text{AHC}}$ and $\mathcal{T}$ here are unordered, so the positions of b and c in $\mathcal{T}$ do not matter as long as they are children of a .

Lemma B.2 establishes the connection between the equi-depth multifurcating tree of Ψ -RAG and the binary tree of AHC.

Lemma B.2 (Connection between AHC and Ψ -RAG). *There exists a surjection $\psi : \mathcal{T}_{\text{AHC}} \mapsto \mathcal{T}$ that maps every binary tree $\mathcal{T}_{\text{AHC}}$ to an equi-depth multifurcating tree $\mathcal{T}$ by iteratively performing node collapsing.*

Proof. The proof is intuitive. First, the merging process of Ψ -RAG is identical to that in an AHC tree: both of them group two nodes under a common parent.

We now focus on collapsing. As illustrated in Figure 7, the process $\psi(\mathcal{T}_{\text{AHC}})$ can be viewed as a bottom-up layer-by-layer traversal process. Whenever a node is found whose subtrees have unequal depths, the root of the shallower subtree is collapsed to the corresponding node in the deepest subtree. This process iterates until every node has subtrees of equal depth. Therefore, any $\mathcal{T}_{\text{AHC}}$ can always be mapped to an equi-depth multifurcating tree $\mathcal{T} = \psi(\mathcal{T}_{\text{AHC}})$. $\square$

Remark. Lemma B.2 allows us to decompose $\psi(\mathcal{T}_{\text{AHC}})$ bottom-up into a sequence of collapsing operations. Combined with Lemma B.1, it implies that $\Delta c = c(\psi(\mathcal{T}_{\text{AHC}})) - c(\mathcal{T}_{\text{AHC}})$ can be decomposed into a layer-by-layer summation of the cost increments introduced by node collapse. Moreover, it is obvious that Dasgupta's cost is invariant to the order of children of a certain node, so Δc is invariant to the order of collapsing within a certain layer as well.

In the following analysis, we follow the original paper (Dasgupta, 2016) by considering a computable scenario where the distance $d = 1$ for all pairs of chunks. The subsequent conclusions demonstrate that this assumption does not affect our conclusion of Ψ -RAG's property. To analyze the non-uniformity preference of Ψ -RAG, Lemma B.3 quantifies the increment in Dasgupta's cost introduced by node collapse.

Lemma B.3 (Increment of Dasgupta's cost of node collapse). *The following holds:*

(1) For a leaf node collapse within a tree $\mathcal{T}$, let the total number of leaves be $|\ell(\mathcal{T})| = n$, and let $\mathcal{T}_a$ be the subtree rooted at the abstract node a to which the leaf node u is being linked, with $|\ell(\mathcal{T}_a)| = n_a$. This introduces an increment of Dasgupta's cost $\Delta c = \binom{n_a}{2}$.

(2) For an abstract node collapse within a tree $\mathcal{T}$, let the number of leaves of the subtree to be collapsed $\mathcal{T}_u$ be n' . Let $\mathcal{T}_a$ be the subtree rooted at the abstract node a to which $\mathcal{T}_u$ is being linked. $\mathcal{T}_a$ has k branches with the number of leaves $n_1, n_2, \dots, n_k$, respectively. This introduces an increment of Dasgupta's cost $\Delta c = n' \sum_{1 \leq i < j \leq k} n_i n_j$.

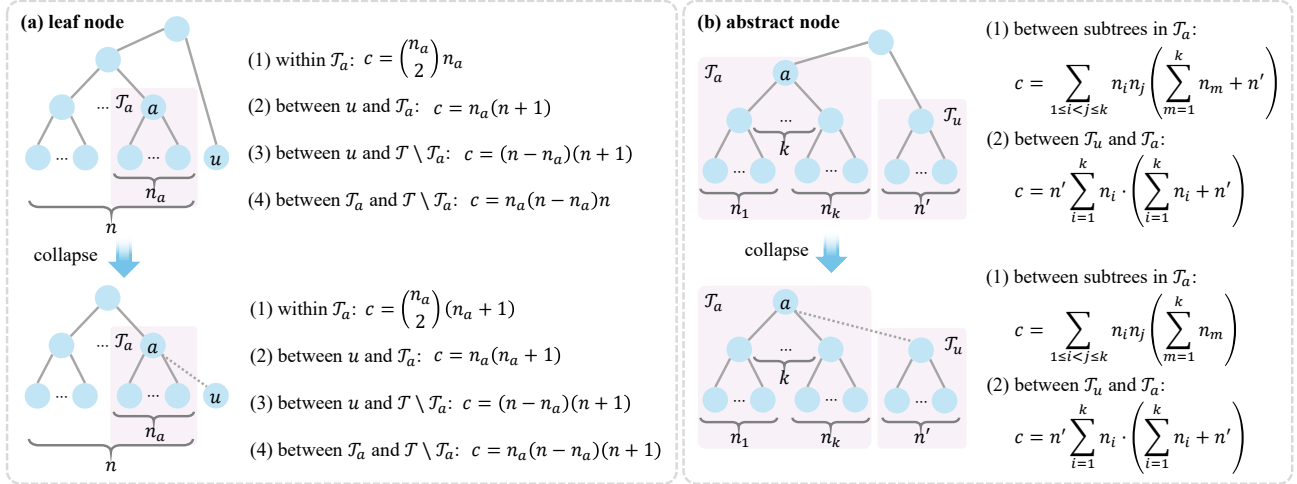


Figure 8. Decomposition of Dasgupta’s cost for (a) leaf node collapse and (b) abstract node collapse. The Dasgupta’s costs of unmentioned components in the figure obviously remain invariant before and after the collapse.

Proof. (1) From Fig 8 (a),

$$\Delta c = \underbrace{\binom{n_a}{2} (n_a + 1 - n_a)}_{\Delta c(\text{within } T_a)} + \underbrace{n_a(n_a + 1) - n_a(n + 1)}_{\Delta c(\text{between } u \text{ and } T_a)} + \underbrace{n_a(n - n_a)(n + 1 - n)}_{\Delta c(\text{between } T_a \text{ and } T \setminus T_a)} = \binom{n_a}{2} \quad (32)$$

(2) From Fig 8 (b),

$$\Delta c = \underbrace{\sum_{1 \leq i < j \leq k} n_i n_j \left(\sum_{m=1}^k n_m + n' - \sum_{m=1}^k n_m \right)}_{\Delta c(\text{between subtrees in } T_a)} = n' \sum_{1 \leq i < j \leq k} n_i n_j \quad (33)$$

which ends the proof. $\square$

Corollary B.4. By the Hockey-stick identity (Jones, 1996), the total cost increment from performing $n - 2$ consecutive leaf collapses on a binary branch (resulting in $n \geq 3$ leaves) is

$$\Delta c = \sum_{i=2}^{n-1} \binom{i}{2} = \binom{n}{3} \quad (34)$$

The total cost increment from collapsing $K - k$ ($K > k \geq 2$) identical subtrees onto node a (resulting in K subtrees) is

$$\Delta c = \sum_{\kappa=k}^{K-1} \left(n' \sum_{1 \leq i < j \leq \kappa} n_i n_j \right) = n' \sum_{\kappa=k}^{K-1} \sum_{1 \leq i < j \leq \kappa} n_i n_j \quad (35)$$

Now we are ready for the following theorem.

Theorem 4.3 (Non-uniformity preference of Ψ -RAG). Consider a 3-layer equi-depth multifurcating tree $\mathcal{T}$ with a perfectly uniform leaf distribution among its subtrees $\mathcal{T}_1, \dots, \mathcal{T}_k$: $n_1 = n_2 = \dots = n_k = n/k$, where $k \geq 3$, $n \geq 3k$. A new tree $\mathcal{T}'$ is constructed by moving one leaf from $\mathcal{T}_j$ to another subtree $\mathcal{T}_i$. Then:

(1) $c(\mathcal{T}') < c(\mathcal{T})$;

(2) Let $\mathcal{T} = \psi(\mathcal{T}_{AHC})$ and $\mathcal{T}' = \psi(\mathcal{T}'_{AHC})$. Denote by Δc and $\Delta c'$ the increments of Dasgupta’s cost on $\mathcal{T}$ and $\mathcal{T}'$ introduced by $\psi(\cdot)$, respectively. Then we have $c(\mathcal{T}') - c(\mathcal{T}) = \Delta c' - \Delta c$.

Proof. (1) We visualize the scenario in Figure 9. Let $m = n/k$ and we can know from Lemma B.1 that

$$c(\mathcal{T}) = k \binom{m}{2} m + \binom{k}{2} m^2 n, \quad (36)$$

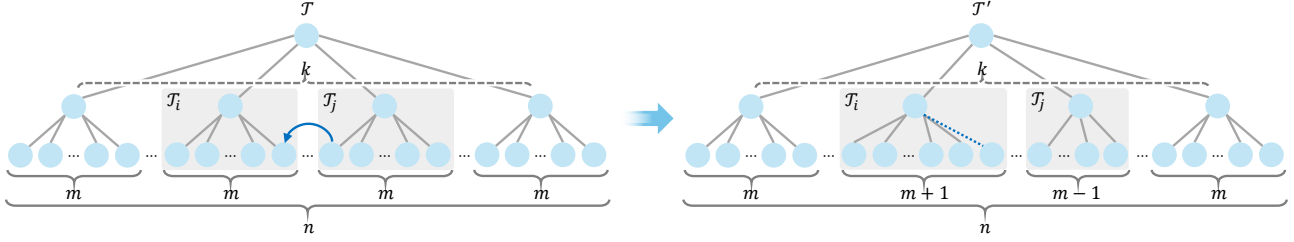


Figure 9. An example of a uniform cluster tree (left) and moving a node from $\mathcal{T}_j$ to another subtree $\mathcal{T}_i$ (right).

$$\begin{aligned}
 c(\mathcal{T}') &= (k-2) \binom{m}{2} m + \binom{m+1}{2} (m+1) + \binom{m-1}{2} (m-1) \\
 &\quad + \underbrace{\binom{k-2}{2} m^2 n}_{0 \text{ if } k=3} + (m+1)(m-1)n + (k-2)(m+1)mn + (k-2)(m-1)mn
 \end{aligned} \tag{37}$$

So

$$c(\mathcal{T}') - c(\mathcal{T}) = 3m - n - 1, \tag{38}$$

which yields $c(\mathcal{T}') < c(\mathcal{T})$ if $k \geq 3$.

(2) According to Lemma B.2, we can decompose $\psi(\cdot)$ bottom-up into multiple leaf node collapses and multiple abstract node collapses. From Corollary B.4, the total cost increments introduced by $\psi(\mathcal{T}_{\text{AHC}})$ and $\psi(\mathcal{T}'_{\text{AHC}})$ are

$$\Delta c = k \binom{m}{3} + m \sum_{i=2}^{k-1} \binom{i}{2} m^2 = k \binom{m}{3} + \binom{k}{3} m^3, \tag{39}$$

$$\begin{aligned}
 \Delta c' &= (k-2) \binom{m}{3} + \underbrace{\binom{m-1}{3}}_{0 \text{ if } m=3} + \binom{m+1}{3} + \underbrace{\binom{k-2}{3} m^3}_{0 \text{ if } k \leq 4} + \underbrace{\binom{k-2}{2} (m-1) m^2}_{0 \text{ if } k=3} \\
 &\quad + (m+1) \left(\underbrace{\binom{k-2}{2} m^2}_{0 \text{ if } k=3} + (k-2)m(m-1) \right)
 \end{aligned} \tag{40}$$

So we have

$$\Delta c' - \Delta c = 3m - n - 1 = c(\mathcal{T}') - c(\mathcal{T}) \tag{41}$$

which ends the proof. $\square$

Remark. Theorem 4.3 proves that while traditional AHC assigns the same Dasgupta's cost to the two different tree structures ($c(\mathcal{T}) - \Delta c = c(\mathcal{T}') - \Delta c'$), the collapsing process in Ψ -RAG results in a lower Dasgupta's cost for the tree with a non-uniform leaf distribution. This indicates that there is no uniform effect for Ψ -RAG even when pairwise distances between leaf nodes are unsorted. Moreover, as the computation of Dasgupta's cost can be decomposed into that of each subtree, our conclusions can be generalized to trees with $\delta > 3$.

B.3. Proof of Theorem 4.4

Theorem 4.4 (Distribution adaptability of Ψ -RAG). *Consider a 3-layer equi-depth multifurcating tree $\mathcal{T}$ with subtrees $\mathcal{T}_1, \dots, \mathcal{T}_k$ of any size where $k \geq 3$. A subtree of a minor leaf cluster $\mathcal{T}_i$ has $2 \leq n_i < n_j, \forall 1 \leq j \neq i \leq k$. A new tree $\mathcal{T}'$ is constructed by moving one leaf from some subtree $\mathcal{T}_j$ to the minor subtree $\mathcal{T}_i$. Then:*

- (1) $c(\mathcal{T}') \geq c(\mathcal{T})$, and $c(\mathcal{T}') = c(\mathcal{T})$ iff $n_j = n_i + 1$;
- (2) Let $\mathcal{T} = \psi(\mathcal{T}_{\text{AHC}})$, $\mathcal{T}' = \psi(\mathcal{T}'_{\text{AHC}})$ and $\Delta c, \Delta c'$ defined as in Theorem 4.3. Then we have $\Delta c' \geq \Delta c$, and $\Delta c' = \Delta c$ iff $n_j = n_i + 1$.

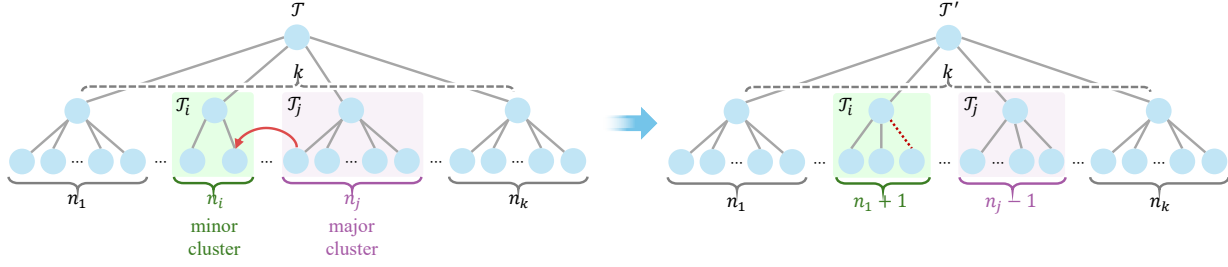


Figure 10. An example of a tree with a minor subtree $\mathcal{T}_i$ (left) and moving a node from a major subtree $\mathcal{T}_j$ to $\mathcal{T}_i$ (right).

Proof. (1) We visualize the scenario in Figure 10. So we have

$$c(\mathcal{T}) = \binom{n_i}{2} n_i + \binom{n_j}{2} n_j + n_i(n - n_i)n + n_j(n - n_j)n + C, \quad (42)$$

$$c(\mathcal{T}') = \binom{n_i + 1}{2} (n_i + 1) + \binom{n_j - 1}{2} (n_j - 1) + (n_i - 1)(n - n_i + 1)n + (n_j - 1)(n - n_j + 1)n + C \quad (43)$$

where C is a constant that remains invariant to the structure change. So

$$c(\mathcal{T}') - c(\mathcal{T}) = 2n(n_j - n_i - 1) + \frac{3n_i^2 + n_i - 3n_j^2 + 5n_j - 2}{2} \quad (44)$$

When $n_j = n_i + 1$, $c(\mathcal{T}') - c(\mathcal{T}) = 0$. When $n_j = n_i + x$ where $x > 1$, then

$$c(\mathcal{T}') - c(\mathcal{T}) = 2n(x - 1) + \frac{3n_i^2 + n_i - 3(n_i + x)^2 + 5(n_i + x) - 2}{2} \quad (45)$$

$$= \left(2n - 3n_i - \frac{3x - 2}{2} \right) (x - 1) \quad (46)$$

$$= \left(2 \left(n_i + n_i + x + \sum_{o \neq i, j} n_o \right) - 3n_i - \frac{3x - 2}{2} \right) (x - 1) \quad (47)$$

$$= \left(n_i + 2 \sum_{o \neq i, j} n_o + \frac{x}{2} + 1 \right) (x - 1) \quad (48)$$

$$> 0 \quad (49)$$

which yields $c(\mathcal{T}') > c(\mathcal{T})$.

(2) From Corollary B.4, the total cost increments introduced by $\psi(\mathcal{T}_{\text{AHC}})$ and $\psi(\mathcal{T}'_{\text{AHC}})$ are

$$\Delta c = \binom{n_i}{3} + \binom{n_j}{3} + (n - n_i - n_j)n_i n_j + C, \quad (50)$$

$$\Delta c' = \binom{n_i + 1}{3} + \underbrace{\binom{n_j - 1}{3}}_{0 \text{ if } n_j = 3} + (n - n_i - n_j)(n_i + 1)(n_j - 1) + C \quad (51)$$

where C is a constant that remains invariant to the structure change. So

$$\Delta c' - \Delta c = (n_j - n_i - 1) \left(n - n_i - n_j - \frac{n_i + n_j}{2} + 1 \right) \quad (52)$$

When $n_j = n_i + 1$, $\Delta c' - \Delta c = 0$. When n_j is larger, $n_j - n_i - 1 > 0$, so we focus on the second part of the right hand side. We consider the minimum case of $n = n_i + n_j + n_j$ and

$$\Delta c' - \Delta c = (n_j - n_i - 1) \left(\frac{n_j - n_i}{2} + 1 \right) > 0 \quad (53)$$

which ends the proof. $\square$

Table 9. Statistics of datasets.

Dataset	Task type	Index scale	#Queries	#Documents	Average #tokens per document
NQ	Single-hop QA	Corpus	1,000	9,633	141.330
PopQA	Single-hop QA	Corpus	1,000	8,676	136.536
HotpotQA	Multi-hop QA	Corpus	1,000	9,811	131.602
2Wiki	Multi-hop QA	Corpus	1,000	6,119	110.436
MuSiQue	Multi-hop QA	Corpus	1,000	11,656	114.516
MultiHop-RAG	Multi-hop QA	Corpus	1,000	609	2346.064
NarrativeQA	Narrative QA	Passage	293	10	62,824.500
∞ -LongBook	Narrative QA	Passage	100	19	214,711.895
QMSum	Summarization	Passage	35	35	14,375.543
WCEP	Summarization	Passage	119	119	9,567.235

 Table 10. Statistics of abstract tree indexes of Ψ -RAG.

Dataset	#Trees	#Average leaf nodes	#Average abstract nodes	Average tree depth	Average compression rate
NQ	1	9,633	3,164	6	82.35%  / 83.06% 
PopQA	1	8,676	2,810	5	83.26%  / 81.87% 
HotpotQA	1	9,811	2,635	6	85.98%  / 85.85% 
2Wiki	1	6,119	1,704	5	82.14%  / 82.89% 
MuSiQue	1	11,656	2,938	6	84.79%  / 83.09% 
MultiHop-RAG	1	16,349	4,377	6	78.94%  / 78.87% 
NarrativeQA	10	411	85	3.5	90.71%  / 98.25% 
∞ -LongBook	19	2,446	488	4.2	84.36%  / 84.31% 
QMSum	35	155	41	3.3	82.19%  / 82.33% 
WCEP	119	109	33	3.2	76.32%  / 76.35% 

Remark. Theorem 4.4 proves that Ψ -RAG incurs a larger cost to assign a major cluster node to a minor cluster. For a minor class in a corpus with cluster $c = \{u | \exists v \in c, d(u, v) < d(u, w) \forall w \notin c\}$, it always form a minor subtree in $\mathcal{T}$ following the similarity ranking and iterative merging & collapse processes of Ψ -RAG. In conclusion, Ψ -RAG tends to preserve the size of minor classes in a skewed corpus. As the computation of Dasgupta’s cost can be decomposed into that of each subtree, our conclusions can be generalized to trees with $\delta > 3$.

C. Detailed Experimental Setup

C.1. Dataset and Index Statistics

Statistics of datasets. We provide the statistics of datasets used in our experiments in Table 9.

Dataset split. We follow RAPTOR’s document splitting strategy, segmenting documents into chunks of approximate 100 tokens without cutting mid-sentence. Notably, we do not re-split datasets composed of similarly sized paragraphs: NQ, PopQA, HotpotQA, 2Wiki, and MuSiQue.

Statistics of abstract trees. We provide the statistics of Ψ -RAG’s abstract tree indexes for each dataset in Table 10. The results show that for corpus-level datasets, the tree depth of Ψ -RAG stabilizes at 5 to 6 layers. This demonstrates Ψ -RAG’s key advantage of generalization ability: it does not require prior assumptions about data, as it naturally uncovers the underlying semantic hierarchy.

C.2. Configurations

Indexing setup. We set the maximum token length of the summative abstract for each abstract node to 100, and the maximum number of keywords of the keyword abstract to 20. We set the maximum number of children nodes to 40. Abstract nodes with excessive children will be split in half in the rebalancing process.

The keyword abstract  is used for Ψ -RAG in all non-comparative experiments in this paper, unless otherwise stated.

Retrieval setup. We mainly follow the multi-step setting of HippoRAG (Jiménez Gutiérrez et al., 2024) for agentic retrieval. For QA tasks, top-10 retrieved passages from both sparse and dense tree retrievers are integrated and reranked to the final top-5 at each retrieval step. We maintain a maximum score list for retrieved document chunks at each step and take top-5 documents from the list for recall@ k evaluation. For multi-hop datasets, we limit the maximum number of retrieval attempts ($i_{\max} + 1$) to be equal to the maximum hop count of user questions: 2 for HotpotQA and 2Wiki, 4 for MuSiQue and MultiHop-RAG. For single-hop datasets (NQ and PopQA), though, we allow one extra retrieval attempt for possible query reorganization. For summarization datasets, we perform single retrieval. The k value is determined by the size of documents: top-20 for QMSum and top-15 for WCEP.

LLM setup. All experiments are conducted on up to 3 NVIDIA GeForce RTX 4090 GPUs with 48GB VRAM each. We use Ollama⁵ and Hugging Face as default LLM sources. We follow the majority of default settings of Ollama and Hugging Face models: for example, the context window is fixed to 4096 tokens. We set the decoding temperature to 0 and the penalty for generating repetitive content to 1.3 for every LLM. For the summarization task, the maximum LLM response token length is determined by the size of documents: 200 for QMSum and 150 for WCEP.

C.3. Instruction Prompts

We show LLM prompts used for generating summative abstract, keyword abstract, and R&A agent below. For single-hop and multi-hop QA, we follow existing works and provide a one-shot in-context example.

LLM Prompt for Summative Abstract

===== {System Instruction} =====

As an advanced document summarization assistant, your task is to summarize the given list of documents, including as many key details and relationships as possible.

- (1) Your answer should always start with “Summary:”. NO UNNECESSARY WORDS: Always directly output the summary instead of useless modifiers like ‘Here is a summary of the given list ...’.
- (2) Be faithful to the original documents. Unnecessary paraphrasing and adding content not mentioned in the original documents are NOT allowed.
- (3) Your summary should be NO MORE THAN {summary_max_length} WORDS. Keep in mind that a longer summary is not always better.

An in-context example is provided below.

===== {In-context example} =====

 User input:
 Hotel Tallcorn
 The Hotel Tallcorn is located in Marshalltown, Iowa...
 The Oberoi Group
 The Oberoi Group is a hotel company with its head office in Delhi...
 Hotel Bond
 Hotel Bond is a historic hotel, built in two stages in 1913 and 1921...
 Mohan Singh Oberoi
 Rai Bahadur Mohan Singh Oberoi (15 August 1898 – 3 May 2002)...
 Ritz-Carlton Jakarta
 The Ritz-Carlton Jakarta is a hotel and skyscraper in Jakarta...

 Assistant response:
 The Hotel Tallcorn in Marshalltown, Iowa, built in 1928, is now an apartment building. It was part of one of the second-largest hotel sales in U.S. history in 1956. Hotel Bond is a historic hotel in Hartford, Connecticut, built in two stages. After facing competition, it went through multiple ownership changes and bankruptcies. It is now a Homewood Suites by Hilton, with its ballroom having reopened in 2001. The Oberoi Group, founded in 1934, is a luxury hotel company based in Delhi. It owns and operates over 30 hotels and two cruise ships. It was founded by Mohan Singh Oberoi, who started India’s second-largest hotel company. The Ritz-Carlton Jakarta is a skyscraper hotel in Indonesia, opened in 2005. It consists of two towers, one for the hotel and one for apartments, and is located near the JW Marriott Hotel.

⁵<https://ollama.com/>

LLM Prompt for Keyword Abstract

===== {System Instruction} =====

As an advanced document summarization assistant, your task is to summarize the given list of text paragraphs into multiple key phrases or words. These key phrases include:

- Key phrases that overview the types of information covered in the text;
- Key phrases that reflect the common themes of these paragraphs;
- Key phrases that distinguish the specific content of each paragraph.

Requirements:

- (1) Generate key phrases mentioned above in order. All key phrases should be strictly formatted as ONE line of text, separated by commas, equivalent to the output format of Python code `"`, ' .join(list_of_keywords)"`.
 - (2) Different key phrases should be distinct from one another.
 - (3) NO UNNECESSARY WORDS: Always output the direct key phrase list instead of useless modifiers like 'Here are the key phrases according to the documents: ...' or 'Information key phrases: ... Theme key phrases: ...'.
 - (4) The number of key phrases should be NO MORE THAN {max_num_keywords}.
- An in-context example is provided below.

===== {In-context example} =====

 User input:

Hotel Tallcorn
 The Hotel Tallcorn is located in Marshalltown, Iowa...
 The Oberoi Group
 The Oberoi Group is a hotel company with its head office in Delhi...
 Hotel Bond
 Hotel Bond is a historic hotel, built in two stages in 1913 and 1921...
 Mohan Singh Oberoi
 Rai Bahadur Mohan Singh Oberoi (15 August 1898 – 3 May 2002)...
 Ritz-Carlton Jakarta
 The Ritz-Carlton Jakarta is a hotel and skyscraper in Jakarta...

 Assistant response:

historic hotels,hotel companies,bankruptcy and renovation,brand operations,Indian hospitality industry,Indonesia skyscraper,hotel locations,hotel building time,hotel architecture,hotel founders,hotel ownership history,Marshalltown Iowa,Hotel Tallcorn,Eppley Hotel Company,Sheraton sale,The Oberoi Group,Mohan Singh Oberoi,Hotel Bond,Hartford Connecticut,Bond Ballroom,Ritz-Carlton Jakarta

LLM Prompt for R&A Agent in single-hop and multi-hop QA

===== {System Instruction} =====

As an advanced reading comprehension and information retrieval agent, your task is to answer the user question by retrieving documents and multi-step reasoning, following the instructions below:

- (1) Your response starts after "Thought: ", where you will methodically break down the reasoning process, illustrating how you arrive at conclusions step-by-step. If you believe you have found the answer, conclude with "Answer: <answer>" to present a concise, definitive response.
 - (2) You can request for retrieving supporting documents from a relevant database a maximum of {max_retrieval_time} times. Some retrieved documents are provided the first time. When the question involves content you do not understand, or current documents do not provide enough information, request retrieval by responding with "Retrieve: <retrieve_query>", where <retrieve_query> is a sub-question derived from the user question, which you believe must be resolved to answer the user question.
 - (3) The retrieval results will be provided in the format "Retrieved documents: ...", and you should continue your reasoning based on the user question, your previous reasoning steps, and all retrieved documents. Obtain your answer with as few retrieval attempts as possible, which means you need to make your <retrieve_query> as precise as possible. If newly retrieved documents do not help, try reorganizing your <retrieve_query>. If you run out of retrieval attempts and still cannot answer the question, answer with "Not mentioned" only. Do NOT provide additional uncertain information.
 - (4) NO UNNECESSARY WORDS: After your reasoning, you MUST end your response with "Answer: <answer>" or "Retrieve: <retrieve_query>" in a new line. <answer> is in most cases a word or a phrase. Avoid ambiguous words like "approximately".
 - (5) Your "Thought: " response should be NO MORE THAN {thought_max_length} WORDS.
- An in-context example is provided below.

===== {In-context example} =====



User input:

Retrieved documents:

Hotel Tallcorn

The Hotel Tallcorn is located in Marshalltown, Iowa...

Hotel Bond

Hotel Bond is a historic hotel, built in two stages in 1913 and 1921...

Ritz-Carlton Jakarta

The Ritz-Carlton Jakarta is a hotel and skyscraper in Jakarta...

User question: The Oberoi family is part of a hotel company that has a head office in what city?

Retrieval time remaining: 3

Thought:



Assistant response:

According to the user question, I need to first figure out the hotel company that the Oberoi family is part of. However, the retrieved documents do not include any details regarding the Oberoi family.

Retrieve: Which hotel company is the Oberoi family part of?



User input:

Retrieved documents:

The Oberoi Group

The Oberoi Group is a hotel company with its head office in Delhi...

Mohan Singh Oberoi

Rai Bahadur Mohan Singh Oberoi (15 August 1898 – 3 May 2002)...

User question: The Oberoi family is part of a hotel company that has a head office in what city?

Retrieval time remaining: 2

Thought:



Assistant response:

According to “Mohan Singh Oberoi”, Mohan Singh Oberoi is the founder and chairman of Oberoi Hotels & Resorts. From “The Oberoi Group”, Oberoi Hotels & Resorts is a brand and its company, The Oberoi Group, is a hotel company with its head office in Delhi.

Answer: Delhi

LLM Prompt for R&A Agent in summarization

===== {System Instruction} =====

As an advanced reading comprehension and information retrieval agent, your task is to answer summative user questions based on retrieved documents. For each document, an [ID] is prepended to indicate the relative order in the document corpus. These documents provide a condensed version of the original long corpus to help you with summative tasks.

Requirements:

- (1) Organize the documents to form a logically clear summary of the main content or plot of the original long corpus.
- (2) Your answer should be comprehensive and coherent. Your summative answer should be an independent paragraph, enabling the user to comprehend the content or plot of the original corpus. Do NOT include additional uncertain or made-up information in your answer.
- (3) Your answer should be NO MORE THAN {response_max_length} WORDS.

D. Supplemental Experiments

D.1. Case Study

We present a multi-hop example from MuSiQue in Table 11 to intuitively illustrate how Ψ -RAG performs multi-granular agentic retrieval on the abstract tree. The question, “*Who is the wife of the man who produced the documentary of the pop star who influenced Beyoncé?*” requires up to 3 hops to answer. The example shows that the keywords retrieved by Ψ -RAG at the first layer are relatively coarse, covering broad themes such as music, sports, and movies. With the search going deeper into the tree, the keywords become finer-grained. Consequently, two semantically related retrieved chunks tell that the required “man who produced the documentary” is David Gest.

However, due to the structural isolation, we still do not know who is David Gest’s wife, so the R&A agent calls for a second retrieval attempt. As the second question lacks sufficient context, the coarse abstraction in the upper tree layers confuses

Hierarchical Abstract Tree for Cross-Document Retrieval-Augmented Generation

Table 11. Retrieved documents and agent response of a multi-hop question example. “✓” and “✗” indicate supporting chunks and irrelevant chunks, respectively. Relevant tokens matched by BM25 are highlighted.

Query		Who is the wife of the man who produced the documentary of the pop star who influenced Beyoncé?	
		Ψ-RAG (1st attempt: original query)	Ψ-RAG (2nd attempt: Who is the spouse of David Gest?)
BM25		✗ 1: “The Mistress” features... a married man , whose wife was played by... ✗ 2: Pizza Man is a 1991 comedy film starring Bill Maher... ✗ 3: Real Life is a 1979 American comedy film starring Albert Brooks... ✗ 4: No Escape is a 1936 British thriller film directed by... ✓ 5: Michael Jackson: The Life of an Icon is a documentary film about pop singer... produced by his friend, David Gest ...	✓ 1: Liza and David ... featuring Liza Minnelli and her then-husband, David Gest . ✓ 2: Michael Jackson: The Life of an Icon... produced by his friend, David Gest .. ✗ 3: Dave Fennoy Fennoy in Phoenix, Arizona David Henderson Fennoy... ✗ 4: Chileab... was the second son of David , King of Israel... ✗ 5: Random Hearts is a 1999 American romantic drama film... their spouses were having an affair...
	1st layer	1: Beyoncé’s music career, NBA players, American football, ... 2: film characters, biographies of actors, musical fantasy comedies, ... 3: DC Comics characters, superhero films, Marvel and DC comics, ...	1: DC Comics characters, superhero films, Marvel and DC comics, ... 2: cricket tournaments, FIFA Confederations Cup, ICC Champions Trophy, ... 3: film characters, biographies of actors, musical fantasy comedies, ...
	Last abstract layer	1: Beyoncé’s music career, R&B and pop genres, female empowerment, ... 2: show business families, celebrity siblings, marriages and spouses, ... 3: Beyoncé’s solo career, music awards, Billboard charts, ...	1: Swedish personalities, biographical accounts, historical figures, ... 2: Game of Thrones, Justice League, Aquaman, ... 3: European royalty, medieval history, noble families, ...
	Leaf layer	✗ 1: Beyoncé’s work has influenced numerous artists... ✗ 2: On March 30, 2015, it was announced that Beyoncé is a co-owner... ✓ 3: Beyoncé names Michael Jackson as her major musical influence... ✓ 4: Michael Jackson: The Life of an Icon is a documentary film... ✗ 5: Beyoncé Giselle Knowles-Carter (/biːˈjɒnsei/ bee-YON-say)...	✗ 1: Dave Fennoy Fennoy in Phoenix, Arizona... ✗ 2: Gal Gadot - Varsano (Hebrew: ... ✗ 3: Gal Gadot - Varsano (Hebrew: ... ✗ 4: Gal Gadot - Varsano (Hebrew: ... ✗ 5: Catharina Wallenstedt was the daughter of Bishop Laurentius...
	Final retrieved chunks	✓ 1: Michael Jackson: The Life of an Icon... [score: 0.406] ✗ 2: Nine Days that Changed the World is a 2010 documentary... [score: 0.101] ✓ 3: Beyoncé names Michael Jackson as her major musical influence... [score: 0.071] ✗ 4: Beyoncé’s work has influenced numerous artists... [score: 0.064] ✗ 5: Following the disbandment of Destiny’s Child... [score: 0.038]	✓ 1: Liza and David... featuring Liza Minnelli and her then-husband, David Gest. [score: 0.980] ✓ 2: Michael Jackson: The Life of an Icon... [score: 0.201] ✗ 3: Gal Gadot - Varsano (Hebrew: ... [score: 0.029] ✗ 4: Gal Gadot - Varsano (Hebrew: ... [score: 0.026] ✗ 5: Gal Gadot - Varsano (Hebrew: ... [score: 0.020]
Agent response		According to “Michael Jackson: The Life of an Icon”, David Gest is the friend and producer of the documentary film about pop singer Michael Jackson... I need more information on who David Gest’s wife was at that time or if he had a wife. <retrieve> Who is the spouse of David Gest?	According to “Michael Jackson: The Life of an Icon”... Lastly, I found that Liza Minnelli was the wife of David Gest at some point in time. <answer> Liza Minnelli ✓

Table 12. An example of minor class retrieval on MultiHop-RAG (S[:50]+E[:5]). The top-1 summative abstract in each layer is listed. Red and purple text relate to the major class “Sports” and the minor class “Entertainment”, respectively.

Query		What was the name of the tunnel where the fatal car crash occurred in The Crown?	
		RAPTOR (Sarathi et al., 2024)	Ψ-RAG
1st layer		Various sports teams face outcomes: Inter Miami eliminated, Italy and Australia in Davis Cup final, Rangers lead Diamondbacks 3-1, Lions beat Packers, 49ers defeat Cowboys, Vikings lose to Broncos and Bears, amidst other results and controversies including Mattel’s inaccurate Cherokee Barbie.	The Crown’s final season covers Prince Charles and Camilla, Princess Diana, and Dodi Fayed. Diana’s death and her sons’ reactions are depicted. Separate stories include Jada Pinkett Smith and Will Smith’s marriage, Mattel’s Barbie honoring Wilma Mankiller, and Dodi Fayed’s past engagement to Kelly Fisher.
	2nd layer	Inter Miami loses 1-0 to FC Cincinnati, eliminating them from MLS playoff contention, despite Lionel Messi’s 35-minute appearance off the bench, as Alvaro Barreal scores late, while other teams like Manchester United struggle in Europe with defensive issues and Chelsea’s Reece James faces an abusive conduct charge.	Season six focuses on Prince Charles and Camilla, Princess Diana and Dodi Fayed. After Diana’s 1997 death, William went missing at Balmoral, returning 14 hours later. Before her death, Diana told William and Harry not to believe papers and planned life changes, but a planned meeting never occurred due to her fatal car crash in Paris.
	Leaf layer	So, our mistakes can never happen in the Champions League, you get punished for it, so we have to learn from that” However, Aston Villa legend Gabby Agbonlahor insists Onana needs to take a significant chunk of the blame for United’s collapse on Wednesday night “I feel sorry for Onana,” Agbonlahor told talkSPORT “There’s a clip of him throwing his gloves to the ground after the game He’s angry with himself	In both the series and in real-life, Diana and Dodi - along with their chauffeur, Henri Paul - were killed on 31 August 1997 when their Mercedes-Benz was pursued by paparazzi and crashed in the Pont de l’Alma tunnel in Paris The mother of two was 36 years old The fatal car crash was recreated in the very first scene of season six
Agent response		The retrieved documents once again do not mention anything about a tunnel or a fatal car crash from “The Crown”. They are still discussing football, specifically Aston Villa and Manchester United. Since I have run out of retrieval attempts and the provided information does not answer the question. <answer> <i>Not mentioned</i> ✗	To answer this user question, I need to identify the location and details regarding a specific event depicted in “The Crown”. According to the retrieved document, Diana, Dodi, their chauffeur Henri Paul were involved in a fatal car crash. This incident occurred when they were pursued by paparazzi while driving through Paris. Their Mercedes-Benz crashed inside the Pont de l’Alma tunnel on 31 August 1997. <answer> Pont de l’Alma ✓

the retriever, leading it to irrelevant chunks. In contrast, sparse retrieval can provide straightforward facts about David Gest by simply searching for documents with the keyword. As a result, the retrieved chunk leads to the final answer “Liza Minnelli”. This demonstrates the importance of synergy between Ψ -RAG’s hierarchical tree index and the multi-granular hybrid retriever, the key to its effectiveness.

D.2. Analyses on Hierarchical Abstract Tree

Case study: retrieval on skewed data. To further demonstrate the impact of the uniform effect and distribution adaptability, we conduct a case study based on the visualization study in Section 4.2. We compare the agent response processing a minor-class question on both tree indexes of RAPTOR and Ψ -RAG. From Table 12, the 1st layer abstract nodes of RAPTOR are confused because of the uniform effect, with most of the contents related to the major class “Sports”. This significantly interrupts the retriever which selects a major class abstract node in the 2nd layer. On the contrary, Ψ -RAG’s minor abstract node at the 1st layer clearly points to the relevant topic “The Crown”. This assists the retriever in finding the correct chunk.

Table 13. Comparative results of the indexes. “SR”, “R&A”, and “Q3R” refer to the sparse retriever, the R&A agent, and the reranker, specifically.

Index	Single retrieval time (s)	F1 (%)
<i>MuSiQue</i>		
Flat	33.39	30.50
GMM	0.16	17.17
HAT	0.15	18.62 ($\uparrow 1.45$)
GMM + SR + R&A + Q3R	1.17	47.26
HAT + SR + R&A + Q3R	1.12	48.90 ($\uparrow 1.64$)
<i>MultiHop-RAG</i>		
Flat	62.21	42.44
GMM	0.17	35.81
HAT	0.16	36.30 ($\uparrow 0.49$)
GMM + SR + R&A + Q3R	0.66	51.32
HAT + SR + R&A + Q3R	0.62	56.23 ($\uparrow 4.91$)

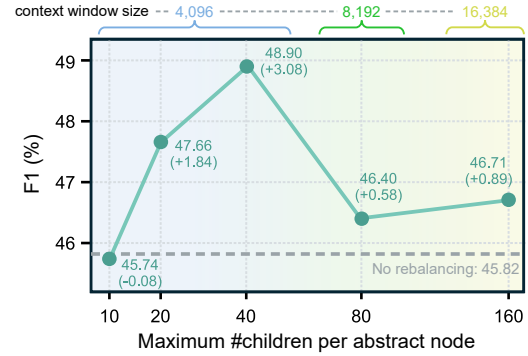


Figure 11. Sensitivity of Ψ -RAG to the maximum number of children per abstract node on MuSiQue.

Quantitative study on tree indexes. We conduct a quantitative comparison between the hierarchical abstract tree (HAT) of Ψ -RAG and (1) a flat index, which performs brute-force sequential search on all document chunks with $O(n)$ time; (2) the GMM-based cluster tree of RAPTOR (Sarathi et al., 2024), both without and with the proposed multi-granular agentic retriever modules. From Table 13, we make the following observations:

(1) **Flat dense retrieval is much slower than tree retrieval.** Although dense retrieval on a flat index serves as a performance upper bound for tree indexes, its linear time complexity incurs at least $\sim 100\times$ higher runtime than the logarithmic retrieval of tree indexes.

(2) **HAT outperforms the GMM tree both with or without the agentic retriever modules.** The performance increase points to the minor class distribution in the data, which characterizes the contribution of our tree index to alleviating poor distribution adaptability.

Effect of rebalancing. We have set the maximum number of children per abstract node for the abstract tree. This keeps the total abstract length within the context window of the abstraction agent. Figure 11 shows the sensitivity of Ψ -RAG to the rebalancing process after tree construction, which lead to three observations. (1) Rebalancing without excessive splitting helps build a better tree structure with higher performance. (2) Splitting an abstract node too many times may lead to relevant node missing during top- k dense vector matching: setting maximum #children to 10 makes the F1 lower than that without rebalancing. (3) When the abstracts become too long (8k–16k tokens), their semantics become more vague which also reduces the accuracy.

D.3. Analyses on Multi-granular Agentic Retriever

Maximum number of retrieval attempts. We evaluate how the efficiency and performance of the R&A agent vary with the maximum number of iterative retrieval attempts ($i_{\max} + 1$), as shown in Figure 12. The results indicate that on multi-hop datasets, the first extra retriever call yields the greatest performance gain, while subsequent attempts contribute minimal or even negative improvement. Each retrieval attempt introduces a relatively constant time overhead, and longer reasoning

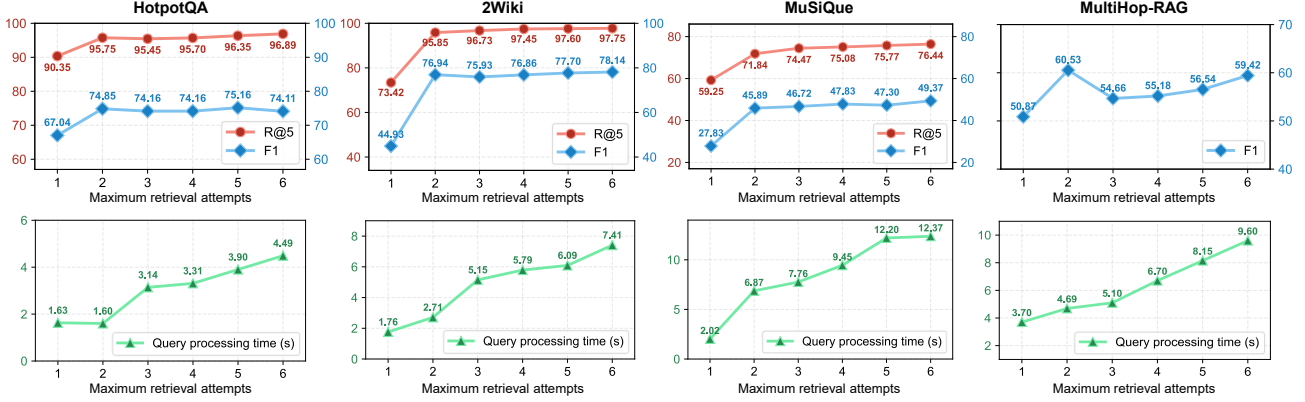


Figure 12. Retrieval and QA performance in Recall@5 (R@5%) and F1 (%), and average processing time for each query (s) under concurrent execution, w.r.t. the maximum number of retrieval attempts.

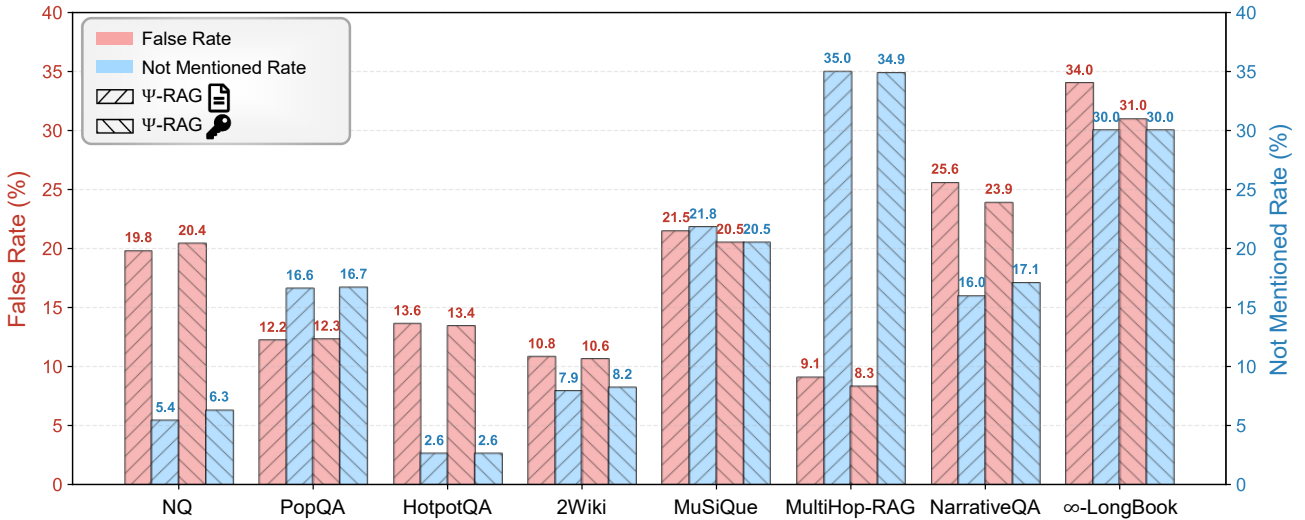


Figure 13. False Rate and Not Mentioned Rate of Ψ -RAG on different datasets.

chains tend to introduce unnecessary noise (Wu et al., 2025). Therefore, it is crucial to strike a balance between the cost of retrieval iterations and the performance gains across different application scenarios.

D.4. Failure Analysis

We analyze the failure cases of Ψ -RAG by measuring the False Rate (rate of incorrect answers) and Not Mentioned Rate (rate of retrieval failures). An incorrect answer is defined as sharing no overlapping tokens with the ground truth. Figure 13 reveals the following key patterns:

(1) **Retrieval failures stem from multiple factors.** Index scale is a primary factor: the Not Mentioned Rate for ∞ -LongBook is significantly higher than that for NarrativeQA, which has an index only 30% the size on average. This is because upper-level abstracts in a larger search space are coarser and introduce more noise. Another factor is query complexity: datasets with higher maximum hop counts (MuSiQue with 4 hops) exhibit higher Not Mentioned Rates than those with fewer hops (HotpotQA and 2Wiki with 2 hops). Although we increase the maximum retrieval attempts for complex questions, we observe that if the initial retrieval for a reorganized query fails, subsequent retrieval attempts are also highly likely to fail. This is because the reorganized query often does not shift the high-level semantic focus, causing the tree retriever to revisit the same abstract nodes. Meanwhile, the high-frequency entities in the reorganized query lead the sparse retriever to return many noisy chunks. As shown in Section D.1, BM25 retrieves many irrelevant chunks with “David” that are unrelated to the target “David Gest”.

(2) **The False Rate consistently exceeds the Not Mentioned Rate.** This occurs on most datasets excluding MultiHop-RAG (with unanswerable negative samples). A manual analysis of 50 failure cases from L3-70B on MuSiQue identifies the following causes: (i) LLM misguidance or guesses resulting from incomplete retrieval (29 cases); (ii) logical errors in agentic reasoning (8 cases); (iii) LLM hallucination (6 cases); (iv) granularity mismatch between the answer and the ground truth (7 cases). Therefore, the primary cause of a higher false rate is missing key document chunks in retrieval.

E. Real-World Extensions

E.1. Approximate Nearest Neighbor Search for Efficient Indexing

We extend Ψ -RAG with two efficient techniques for very large corpora with 10M+ and even 100M+ tokens. They include:

- **Bucketing:** following the inverted-file index in Faiss (Douze et al., 2025), we first partition a large corpus into multiple buckets using a fast recursive spherical k -means. We then build a tree inside each bucket and finally merge them at the highest layer. Assuming $b \ll n$ buckets of balanced size, bucketing reduces the average indexing time complexity from $O(n^2)$ to $O(n^2/b)$ and space complexity from $O(n^2)$ to $O(n^2/b^2)$.
- **ANN search with HNSW** (Malkov & Yashunin, 2018): we build an HNSW graph over all chunk embeddings and use the top- k graph neighbor pairs of each chunk, which are then ranked to build the tree. This reduces the average time complexity of similarity ranking from $O(n^2 \log n)$ to $O(kn \log kn)$ and space complexity from $O(n^2)$ to $O(kn)$.

We evaluate the efficiency of the two extensions on two large subsets from Wiki18 (~ 3 B tokens) in FlashRAG (Jin et al., 2025c): Wiki18-6M (6.1M tokens, ~ 42 k chunks) and Wiki18-50M (50.8M tokens, ~ 350 k chunks). The time and memory costs are listed in Table 14. It shows that HNSW can generate sorted document pairs for a corpus of 50M tokens with less than 1.5 GB of storage. Combined with bucketing, tree indexing can be completed with MB-level memory usage and at less than 1% of the original time cost. Therefore, LLM-based abstraction becomes the primary efficiency bottleneck for scaling the framework to industrial corpora. Exploring efficient alternatives to LLM-based abstraction will be a key direction for our future work.

Table 14. Time and memory comparison of Ψ -RAG’s extensions on large-scale Wiki18 subsets. Embedding dimension is set to 1,024. Experiments are conducted on 8 CPU cores (Wiki18-6M) / 64 CPU cores (Wiki18-50M) and 256GB Memory.

Data	Wiki18-6M			Wiki18-50M	
	Vanilla Ψ -RAG	+ HNSW	+ Bucketing ($b = 4$) + HNSW	+ HNSW	+ Bucketing ($b = 63$) + HNSW
Embedding time (s)	1,229			10,208	
Similarity ranking time (s)	304.84	55.99 (5.44 $\times$)	0.89 (342.5 $\times$)	168.02	7.87 (21.35 $\times$)
Similarity matrix memory	13.16GB	10.55MB (1,277 $\times$)	2.45MB (5,483 $\times$)	88.17MB	2.15MB (41.01 $\times$)
Similarity ranking memory	65.81GB	656.7MB (102.6 $\times$)	37.89MB (1,778 $\times$)	1.34GB	34.10MB (40.24 $\times$)
Tree indexing time (s)	15,996	13,957 (1.15 $\times$)	848 (18.86 $\times$)	$\sim 280,000$	2,261 ($\sim 124\times$)
Abstraction time (s)	$\sim 205,000$	$\sim 200,000$	$\sim 89,400$	$\sim 1,800,000$	$\sim 1,400,000$

E.2. Query Hop Discriminator

Our evaluation utilizes the maximum hop count of each dataset for the maximum number of iterative retrieval time $i_{\max}$. This limits the applicability in real-world datasets with diverse user queries. For practical settings without prior maximum hop count, we implement a lightweight query hop discriminator $\mathcal{Q} : q \mapsto i_{\max}$ using a 2-layer MLP trained on questions with 1 to 4 hops. It predicts the hops of each user query before retrieval to automatically decide the maximum retrieval time for the R&A agent with very small latency. Table 15 shows the QA results with $\mathcal{Q}$ on some multi-hop datasets⁶. Results indicate that the query hop discriminator preserves strong effectiveness of Ψ -RAG without the prior maximum hop count.

Table 15. Multi-hop QA results of Ψ -RAG with the query hop discriminator.

Dataset	R@5 (%)	F1 (%)
HotpotQA	95.05	74.27
MuSiQue	72.68	46.63

⁶We train $\mathcal{Q}$ with user questions from datasets other than the indexed corpus: NQ, 2Wiki, MultiHop-RAG without negative samples, and MuSiQue for indexing HotpotQA (HotpotQA for indexing MuSiQue). The lengths of decomposed sub-query lists or the number of evidence passages serve as labels.

F. Related Work

Retrieval-Augmented Generation (Lewis et al., 2020; Fan et al., 2024). Based on the type of index and retriever, RAG frameworks can be categorized as: (1) *Sparse retrieval*: methods such as TF-IDF (Sparck Jones, 1972) and BM25 (Robertson et al., 2009) retrieve documents where keywords from the user question appear more often in the corpus. They are fast and suitable for token-level detailed search but less capable of capturing deep semantic meanings. (2) *Dense vector retrieval* (Karpukhin et al., 2020): this approach utilizes powerful language embedding models (Khattab & Zaharia, 2020; Izacard et al., 2022; Lee et al., 2025; Zhang et al., 2025b) to map queries and documents into a continuous vector space (Fan et al., 2024). It retrieves the top- k documents most relevant to the user question using vector metrics like cosine similarity. Sometimes, embedding-based rerankers (Xiao et al., 2024; Zhang et al., 2025b) are also utilized to rerank retrieved documents based on user preference, further improving retrieval quality. Dense vector-based RAG excels at capturing deep semantics and key features of entities, making it suitable for more complex QA scenarios. Nonetheless, dense retrieval could prefer documents highly relevant in features but totally unrelated to target entities. (3) *Hybrid retrieval*: some approaches integrate both sparse and dense retrieval to combine their strengths, leading to a number of powerful RAG frameworks (Gao et al., 2021; Lee et al., 2023; Arivazhagan et al., 2023; Zhang et al., 2024b).

Agentic RAG. The rapid advancement in the intelligence of LLMs has heightened focus on their reasoning capabilities and demands on RAG frameworks. Recent efforts (Yao et al., 2022; Trivedi et al., 2023; Asai et al., 2024; Li et al., 2025c) incorporate specifically designed LLM agents to integrate retrieval into LLM reasoning process. Specifically, the agent recursively decomposes a question into multiple sub-questions and invokes the retriever at each step (Li et al., 2025d). This enables the LLM to autonomously and flexibly handle multi-hop questions. However, most of them are built upon flattened linear or inverted indexes for token-level information.

Graph-RAG. As the most popular form of structured RAG, knowledge graph indexes (Edge et al., 2024; Peng et al., 2025; Xiao et al., 2025) can capture semantic relationships between document chunks or entities. They often build a unified index for an entire corpus using named entity recognition and extraction methods like the Open Information Extraction (OpenIE) (Banko et al., 2007). Cross-document multi-hop associations are then captured by walking from node to node (Jiménez Gutiérrez et al., 2024; 2025), extracting shortest paths (Huang et al., 2025; Chen et al., 2026), or extracting subgraphs (Guo et al., 2025; Li et al., 2025a). However, graph indexes are less effective at capturing hierarchy. Some Graph-RAG methods have to resort to less efficient compensatory strategies to handle summative tasks, including but not limited to hierarchical graphs (Malkov & Yashunin, 2018; Huang et al., 2025), community summaries (Edge et al., 2024; Wang et al., 2026), LLM-based hierarchical filtering (Guo et al., 2025), or graph neural network-based message aggregation (Zhang et al., 2025a). Coupled with the significant overhead of OpenIE, efficient index construction remains a key challenge for Graph-RAG.

Tree-RAG. Tree indexes are introduced to explicitly model the hierarchical relationships of documents while achieving fast retrieval with logarithmic complexity (Liu, 2022). Traditional tree indexes for nearest neighbor search include top-down splitting (Bentley, 1975; Omohundro, 1989) and bottom-up hierarchical clustering (Florek et al., 1951; Sokal & Michener, 1958; Jardine & van Rijsbergen, 1971). However, traditional algorithms offer limited generalization. Most existing dense vector-based Tree-RAG methods (Liu et al., 2021; Liu, 2022; Jin et al., 2025b; Zhao et al., 2025; Fatehkia et al., 2024; Tao et al., 2025) rely on explicit document structure (session titles, tables of contents, etc.) or contextual relationships between adjacent text chunks. As a result, they are not directly applicable to documents lacking explicit structure or to cross-document retrieval. RAPTOR (Sarathi et al., 2024) is the first passage-level Tree-RAG framework designed for unstructured long documents. It recursively clusters document chunks via Gaussian Mixture Model (GMM) and generates a summary for each cluster. MemTree (Rezazadeh et al., 2025) further adapts Tree-RAG to the long-term LLM memory scenario, but both of them are limited to the passage-level indexes and the single-document retrieval setting. To our knowledge, the only corpus-level Tree-RAG is LATTICE (Gupta et al., 2025), which performs beam search retrieval using an LLM-evaluated path relevance score. However, there are no discussions towards the limitations on multi-hop QA, and its tree building process is not open-sourced at the time of writing.

G. Discussion and Future Work

Updating the index. Real-time document insertion and index updates are common requirements for RAG applications, yet they pose challenges for traditional Tree-RAG methods, as adding new documents necessitates re-clustering the entire corpus. Our abstract tree index is constructed based on local pairwise similarities, making it feasible to be updated with new documents.

We propose an easy-to-implement approach for the insertion of Δn new document chunks. A single new node is collapsed to the leaf layer by searching for its most similar leaf in the tree via dense vector matching. Then, for keyword abstracts, Ψ -RAG traverses upstream to find the parent abstract nodes of the new node (or maintains an $O(n)$ parent index table for faster lookups) and appends new keywords to each parent. For summative abstracts, the abstraction LLM is invoked to regenerate the abstract for each parent node, similar to existing abstract updating strategies (Rezazadeh et al., 2025). This costs up to $O(n\Delta n)$ time. For batch insertion of Δn chunks, a tree can first be built with only new chunks and then integrated into the main branch via abstract node collapse. If it is collapsed into the l th layer with n_l nodes, then this updating costs $O((\Delta n)^2 n_l \log \Delta n)$ which is much smaller than that of re-indexing.

We note that repeatedly inserting isolated nodes under the same parent may amplify the semantic or granularity biases between them. Considering the semantic coherence in the real-world textual data, we thus recommend inserting new contents as a subtree rather than as individual nodes when they differ substantially from the existing tree.

Future work. An efficiency challenge for Ψ -RAG and other passage-level Tree-RAG lies in the abstraction LLM calls during indexing. Developing a more efficient abstraction scheme is a key direction for scaling Tree-RAG to industrial applications. Moreover, as analyzed in Section D.4, sparse retrievers like BM25 resort to the traditional named entity recognition technique, neglecting the high co-occurrence frequency of named entities. Another future work involves adapting agents to more specialized and diverse user needs with various post-training techniques, such as reinforcement learning and rejection sampling-based supervised fine-tuning.